

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
School of Computer Science and Engineering



**DEVELOPMENT OF AN INTELLIGENT WEB-BASED
FRAMEWORK FOR EARLY DETECTION OF
BURNOUT IN THE INDUSTRIAL WORKFORCE**

*A Microservice Approach Using the Copenhagen Burnout Inventory
and Random Forest Regression*

By

Nguyen Phuc Dat

Student ID: ITITIU22028

Instructor

Dang Tam Nhan

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Science in Computer Science

Ho Chi Minh City, Vietnam

2026

**DEVELOPMENT OF AN INTELLIGENT WEB-BASED FRAMEWORK FOR
EARLY DETECTION OF BURNOUT IN THE INDUSTRIAL WORKFORCE**

APPROVED BY:

Dang Tam Nhan, Supervisor

Committee Member

Committee Member

THESIS COMMITTEE

ACKNOWLEDGMENTS

First and foremost, I would like to express my sincere gratitude to my supervisor, **Dang Tam Nhan**, for the continuous guidance, constructive feedback, and encouragement throughout this thesis project. Your expertise in artificial intelligence and software engineering greatly shaped the direction of this work, and your patience during every revision cycle made this thesis possible.

I am grateful to the faculty of the **School of Computer Science and Engineering** at **International University – Vietnam National University Ho Chi Minh City** for providing an academic environment that encouraged independent research and critical thinking. The knowledge and skills I gained throughout my undergraduate years formed the foundation upon which this project was built.

I would also like to thank my fellow students and friends for the discussions, late-night study sessions, and mutual support. Special thanks to **Tran Luu Hong Phuong** and **Luong Quang Huy**, whose feedback during the early prototype stages helped identify usability issues and architectural improvements I would not have noticed alone.

I am deeply thankful to my family for their unwavering love and encouragement throughout my studies. Their sacrifices and constant belief in me gave me the strength to persevere through the most challenging phases of this project.

Finally, I acknowledge the open-source community and the developers behind the tools and frameworks used in this thesis, including **React**, **Next.js**, **Node.js**, **Express**, **FastAPI**, and **scikit-learn**, without which building a system of this scope would not have been feasible.

Nguyen Phuc Dat

Ho Chi Minh City, May 7, 2026

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivating Scenario and Use Cases	2
1.3	Problem Statement	3
1.4	Scope and Objectives	4
1.5	Research Questions	5
1.6	Assumptions and Proposed Solution	5
1.7	Thesis Structure	7
2	Literature Review	9
2.1	Occupational Burnout	9
2.1.1	Definition and Impact on the Industrial Workforce	9
2.1.2	Burnout Measurement Instruments	10
2.1.3	The Copenhagen Burnout Inventory in This Thesis	10
2.2	Machine Learning for Health Prediction	11
2.2.1	Supervised Regression	11
2.2.2	Linear Regression	11
2.2.3	Random Forest Regression	12
2.2.4	Model Evaluation Metrics	13
2.3	Web Application Architectures	13
2.3.1	Microservices vs. Monolithic Architecture	13
2.3.2	RESTful API Design	14
2.4	Related Technologies	15
2.4.1	React and Next.js for Frontend Development	15

2.4.2	Node.js and Express for Backend Services	15
2.4.3	FastAPI for Machine Learning Serving	16
2.5	Related Work	16
3	Methodology and System Design	18
3.1	System Architecture Overview	18
3.2	Database Design	19
3.2.1	MySQL Schema	19
3.2.2	MongoDB Collections	20
3.3	Backend Design	21
3.3.1	Layered Architecture	21
3.3.2	Authentication and Authorisation	23
3.3.3	Quiz Scoring Pipeline	23
3.3.4	Alert Rules	24
3.4	AI Microservice Design	24
3.4.1	Dataset Description	24
3.4.2	Feature Selection Justification	25
3.4.3	Data Preprocessing	25
3.4.4	Model Selection	26
3.4.5	FastAPI Service Architecture	27
3.5	Frontend Design	28
3.5.1	Technology Stack and Design Decisions	28
3.5.2	Route Structure and Navigation	28
3.5.3	Design System	29
3.5.4	Data Flow Pattern	30

4	Implementation and Results	32
4.1	Development Environment	32
4.2	Backend Implementation	32
4.2.1	Project Structure and Configuration	32
4.2.2	Authentication Flow Implementation	33
4.2.3	Quiz Scoring Implementation	33
4.2.4	Dashboard Aggregation	34
4.2.5	Alert System Implementation	35
4.2.6	API Endpoint Summary	35
4.3	AI Service Implementation	36
4.3.1	Model Training Results	36
4.3.2	FastAPI Prediction Endpoint	39
4.3.3	Integration with Express Backend	40
4.4	Frontend Implementation	40
4.4.1	Authentication Pages	40
4.4.2	Employee Portal	41
4.4.3	Admin Portal	45
4.4.4	Reusable UI Components	50
4.4.5	State Management with TanStack Query	50
5	Discussion and Evaluation	51
5.1	Model Performance Analysis	51
5.2	Functional Testing	52
5.3	System Feature Evaluation	54
5.4	Answering the Research Questions	55

5.5	Comparison with Related Work	56
6	Conclusion and Future Work	58
6.1	Conclusion	58
6.2	Contributions	59
6.3	Achievement of Objectives	59
6.4	Limitations	60
6.5	Future Work	61
	List of References	63
	Appendices	67
	Appendix A: Complete API Reference	67
	Appendix B: Database Schema	70
	Appendix C: CBI Questionnaire	76

List of Tables

1	Comparison with related work.	17
2	Dataset columns used for model training.	25
3	Primary design tokens.	30
4	Development tools and versions.	32
5	Summary of backend API endpoints.	36
6	Model performance comparison on the test set.	37
7	Functional test cases and results.	52
8	Evaluation of system objectives.	54
9	Complete API endpoint reference.	67

List of Figures

1	Structure of the thesis.	8
2	High-level system architecture.	18
3	Entity-relationship diagram of the MySQL database.	19
4	Layered architecture of the Express backend.	22
5	Quiz submission scoring pipeline.	23
6	Architecture of the Python/FastAPI AI microservice.	27
7	Application route structure and navigation flow.	29
8	Random Forest feature importance scores.	37
9	Distribution of Burn Rate values in the training dataset.	38
10	Pearson correlation heatmap of features and Burn Rate.	38
11	Actual vs. predicted burn rate for both models.	39
12	Residual analysis for the Random Forest model.	39
13	Login page with split-screen layout.	41
14	Registration page.	41
15	Employee dashboard with burn rate gauge and trend chart.	42
16	CBI assessment page — Step 1: Personal Burnout.	43
17	Assessment results page with score cards and support resources.	44
18	Quiz submission history with expandable response detail.	45
19	Admin command centre dashboard.	46
20	Department detail — Employees tab with filters and sorting.	47
21	Department detail — Analytics tab with burnout and KPI visualisations.	47
22	Admin employee detail with burnout and KPI analysis.	48
23	Admin alerts inbox with filtering.	49

24	Help resource management with CRUD operations.	49
----	--	----

ABSTRACT

Occupational burnout is a growing concern in industrial workplaces, where high workloads, repetitive tasks, and shift-based schedules place sustained psychological pressure on employees. Traditional approaches to burnout detection rely on periodic surveys and manager observation, which are reactive rather than preventive. By the time symptoms become visible, productivity losses and health consequences have often already occurred.

This thesis presents the design and implementation of an intelligent web-based framework for the early detection of employee burnout in industrial settings. The system combines a validated psychometric instrument—the **Copenhagen Burnout Inventory (CBI)**—with a **Random Forest** regression model to predict a continuous burn rate score from three features: employee designation, resource allocation, and a mental fatigue score derived from the CBI quiz responses.

The framework follows a microservice architecture comprising three components. A **React/Next.js** frontend provides two role-based portals: an employee portal for self-assessment, trend tracking, and access to support resources, and an admin portal with department-level heatmaps, risk distribution charts, KPI analytics, and an automated alert inbox. A **Node.js/Express** backend manages authentication, quiz scoring, risk classification, alert generation, and dashboard aggregation, backed by **MySQL** for relational data and **MongoDB** for the quiz bank and raw submissions. A **Python/FastAPI** microservice loads the trained model at startup and serves predictions via a RESTful endpoint, decoupling the machine learning concern from the application backend.

The Random Forest model, trained on approximately 22,000 records from a public Kaggle dataset, achieves a coefficient of determination (R^2) of approximately

0.90, a mean absolute error (MAE) of approximately 0.048, and a root mean squared error (RMSE) of approximately 0.061 on the held-out test set, outperforming the Linear Regression baseline across all three metrics. Feature importance analysis confirms that the mental fatigue score is the dominant predictor, accounting for over 80% of the model's predictive power.

Several limitations remain. The HR profile data used for designation and resource allocation is simulated rather than sourced from a real factory system. The model uses only three features, and the evaluation relies on a single public dataset that may not generalize across all industrial domains. No formal usability study was conducted. Despite these constraints, the system demonstrates a viable architecture for integrating psychometric assessment, machine learning prediction, and organizational analytics into a unified platform for proactive burnout management.

Keywords: employee burnout detection, Copenhagen Burnout Inventory, Random Forest regression, microservices architecture, industrial workforce, web-based health monitoring.

CHAPTER I

INTRODUCTION

1.1 Background

The industrial workforce operates under conditions that are uniquely conducive to occupational burnout. Factory employees routinely face physically demanding tasks, rigid shift schedules, production quotas, and limited autonomy over their working conditions. The World Health Organization recognised burnout as an occupational phenomenon in the eleventh revision of the International Classification of Diseases (ICD-11), defining it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1]. The three defining dimensions of burnout—emotional exhaustion, depersonalisation, and reduced personal accomplishment—manifest in measurable declines in productivity, increased absenteeism, higher turnover rates, and elevated healthcare costs [2].

Global estimates suggest that stress-related conditions, including burnout, cost businesses hundreds of billions of dollars annually through lost productivity and medical expenses [6]. In manufacturing and industrial sectors, the consequences extend beyond individual well-being: fatigued workers are more prone to safety incidents, quality defects, and equipment damage. Despite these risks, most organisations still rely on ad-hoc observation and infrequent surveys to gauge employee mental health, an approach that is inherently reactive rather than preventive.

Advances in machine learning (ML) and web technologies have created an opportunity to shift burnout management from reactive response to proactive detection.

By combining validated psychometric instruments with predictive models and real-time dashboards, it is now feasible to build systems that continuously assess burnout risk at both the individual and organisational level. Such systems can alert management to emerging problems before they escalate, while simultaneously empowering employees with self-monitoring tools and timely access to support resources.

1.2 Motivating Scenario and Use Cases

Consider a mid-sized manufacturing plant employing 200 workers distributed across six departments, operating on rotating day and night shifts. The plant's HR team consists of three staff members who are responsible for monitoring attendance, handling grievances, and managing benefits. Under current practice, identifying a worker at risk of burnout depends on a supervisor noticing visible signs of distress—chronic fatigue, irritability, or declining output—and escalating the concern to HR. By the time this occurs, the worker may have already experienced weeks or months of deteriorating mental health, and the department may have absorbed measurable productivity losses.

This thesis envisions a system that transforms this reactive workflow into a proactive one. Two primary use cases motivate the design:

1. **Employee self-assessment and trend tracking.** Each worker periodically completes a short 13-question burnout quiz on a web-based portal. The system immediately calculates a predicted burn rate using a machine learning model and presents the result through an intuitive gauge visualisation. Historical trends are displayed as line charts, enabling the worker to observe how their burnout risk evolves over time. If the predicted risk is high or critical, the system surfaces curated mental health resources directly on the results page.
2. **HR department-level monitoring and alerts.** The administrative portal ag-

gregates assessment data across the entire workforce. A department heatmap visualises average burn rates by organisational unit, allowing HR to identify which departments are under the greatest strain at a glance. Automated alerts notify HR when a worker's burn rate crosses a critical threshold or spikes sharply between consecutive assessments. Drill-down views expose anonymised employee-level detail, preserving privacy while enabling targeted intervention.

1.3 Problem Statement

Despite the recognised impact of burnout on industrial productivity and worker welfare, several gaps persist in current practice:

1. **Detection is reactive.** Most organisations identify burnout only after its effects—absenteeism, errors, resignations—have already materialised. There is no systematic mechanism for early warning.
2. **Psychometric data is underutilised.** Validated burnout instruments such as the Copenhagen Burnout Inventory (CBI) exist and are freely available, yet they are rarely integrated into continuous monitoring workflows. When surveys are conducted, the results are typically analysed manually and in isolation.
3. **No predictive capability.** Traditional approaches report current symptoms but do not predict future risk. Machine learning can bridge this gap by learning patterns from historical data and estimating burn rates from a small number of measurable inputs.
4. **No organisational-level visibility.** Individual survey results remain siloed. HR managers lack aggregated, visual tools such as department heatmaps, risk

distribution charts, and trend analyses that would enable data-driven workforce management decisions.

This thesis addresses these gaps by designing and implementing an integrated web-based framework that combines psychometric assessment, machine learning prediction, and multi-level analytics within a single platform.

1.4 Scope and Objectives

The scope of this thesis is the design, implementation, and evaluation of a desktop web application for early burnout detection in industrial workplaces. The system uses simulated HR data (since real factory records are private and inaccessible), the Copenhagen Burnout Inventory as the psychometric instrument, and a Random Forest regression model trained on a public dataset as the predictive engine.

The specific objectives are:

1. Implement a web-based Copenhagen Burnout Inventory (CBI) assessment that converts employee responses into a mental fatigue score suitable for machine learning input.
2. Train and evaluate a Random Forest regression model to predict employee burn rate from three features: designation, resource allocation, and mental fatigue score.
3. Build an employee self-monitoring portal with burn rate gauge, historical trend charts, section score breakdowns, and contextual access to mental health resources.
4. Build an administrative command centre with department heatmaps, risk distribution analytics, KPI correlation views, and a drill-down capability for

individual employee analysis.

5. Implement an automated alert system that notifies administrators when employees cross critical risk thresholds or exhibit sharp burn rate increases between assessments.
6. Demonstrate a microservice architecture that cleanly separates the machine learning inference concern (Python/FastAPI) from the application backend (Node.js/Express) and the user interface (React/Next.js).

1.5 Research Questions

This thesis investigates the following research questions:

- RQ1.** How effectively can a Random Forest regression model predict employee burn rate using only three input features (designation, resource allocation, and mental fatigue score)?
- RQ2.** Can a web-based system provide actionable burnout insights to both employees (through self-monitoring tools) and HR administrators (through department-level analytics and automated alerts)?
- RQ3.** What are the practical benefits and trade-offs of using a microservice architecture to separate machine learning inference from the application backend in a health monitoring system?

1.6 Assumptions and Proposed Solution

The following assumptions underlie the design of the system:

- **Simulated HR data.** Real factory HR databases are private and inaccessible for academic research. The backend generates synthetic HR profiles (designa-

tion, resource allocation, department, shift type) at registration time, drawing values from the same ranges as the training dataset to ensure the ML model receives in-distribution inputs.

- **CBI as a valid burnout measure.** The Copenhagen Burnout Inventory is an established, freely available instrument with demonstrated reliability and validity in occupational health research. Its 13-question excerpt (personal and work-related burnout sections) is used as the sole actively collected data point.
- **Desktop-only usage.** The system targets HR office terminals and factory kiosk stations. No mobile-responsive layout is implemented.
- **Single-language interface.** The user interface is presented in English. Multi-language support is deferred to future work.

The proposed solution is a three-tier web application:

- A **React/Next.js frontend** with two role-based portals (employee and admin), connected to the backend via RESTful API calls, with TanStack Query managing server state.
- A **Node.js/Express backend** that handles authentication (JWT), quiz scoring (CBI with reverse scoring), risk classification, alert generation, dashboard aggregation, and resource management, backed by MySQL and MongoDB.
- A **Python/FastAPI AI microservice** that loads a pre-trained Random Forest model at startup and exposes a single prediction endpoint. The backend can switch between the real AI service and an in-process mock formula via an environment variable, enabling independent development and deployment of each component.

1.7 Thesis Structure

This thesis is organised into six chapters, as illustrated in Figure 1.

Chapter I: Introduction presents the background, motivating scenario, problem statement, objectives, research questions, and assumptions.

Chapter II: Literature Review surveys the academic foundations of occupational burnout, the Copenhagen Burnout Inventory, machine learning methods for health prediction, web application architectures, and related work.

Chapter III: Methodology and System Design describes the overall system architecture, database design, backend design, AI microservice methodology, and frontend design decisions.

Chapter IV: Implementation and Results details the implementation of each component, presents the machine learning training results, and documents the frontend with annotated screenshots.

Chapter V: Discussion and Evaluation analyses model performance, reports functional testing outcomes, evaluates system features against the stated objectives, answers the research questions, and compares the system with related work.

Chapter VI: Conclusion and Future Work summarises the contributions, assesses the achievement of objectives, discusses limitations, and proposes directions for future development.

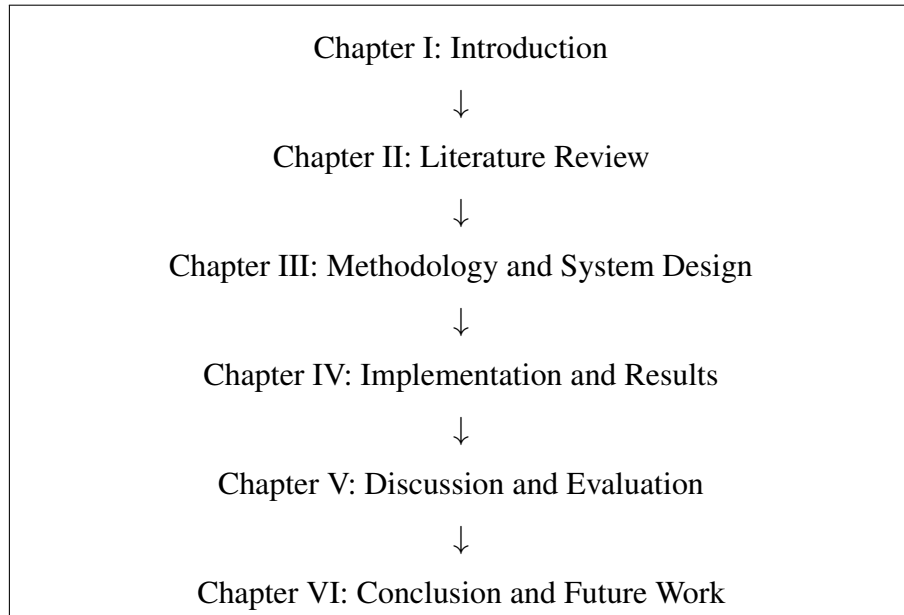


Figure 1: Structure of the thesis.

CHAPTER II

LITERATURE REVIEW

2.1 Occupational Burnout

2.1.1 Definition and Impact on the Industrial Workforce

The concept of occupational burnout was first introduced by Freudenberger in 1974 to describe the state of physical and emotional depletion observed among human service workers [4]. The construct was later formalised by Maslach and Jackson, who defined burnout as a three-dimensional syndrome comprising emotional exhaustion, de-personalisation (cynicism), and reduced personal accomplishment [3]. In 2019 the World Health Organization included burnout in the ICD-11 as an occupational phenomenon, describing it as a syndrome resulting from chronic workplace stress that has not been successfully managed [1].

In industrial and manufacturing environments, the consequences of burnout extend beyond individual suffering. Research has linked burnout to increased absenteeism, higher employee turnover, reduced product quality, and elevated workplace accident rates [7]. A Gallup meta-analysis across industries found that employees experiencing burnout are 63% more likely to take a sick day, 2.6 times more likely to seek a new job, and 13% less confident in their performance [6]. For factory settings with tight production schedules and safety-critical processes, these figures translate directly into operational and financial risk.

2.1.2 Burnout Measurement Instruments

The two most widely used instruments for measuring occupational burnout are the Maslach Burnout Inventory (MBI) and the Copenhagen Burnout Inventory (CBI).

The **Maslach Burnout Inventory (MBI)**, developed in 1981 [3], is the most cited burnout measure in the literature. It consists of 22 items across three subscales (emotional exhaustion, depersonalisation, and personal accomplishment). However, the MBI is copyrighted and requires a commercial licence for use, which limits its adoption in academic and open-source projects.

The **Copenhagen Burnout Inventory (CBI)**, developed by Kristensen et al. in 2005 [5], was created as a freely available alternative. The CBI conceptualises burnout as a state of physical and psychological fatigue and exhaustion, measured along three dimensions: personal burnout, work-related burnout, and client-related burnout. Each dimension is assessed through a set of questions scored on a five-point frequency scale ranging from “Always” (100 points) to “Never/almost never” (0 points). The CBI has demonstrated high internal consistency (Cronbach’s alpha exceeding 0.85 across all subscales) and strong predictive validity for outcomes such as sickness absence and sleep disturbance [5].

2.1.3 The Copenhagen Burnout Inventory in This Thesis

This thesis adopts the CBI for several reasons. First, the CBI is completely free to use, with no licensing restrictions, making it ideal for an academic project. Second, the CBI distinguishes between personal and work-related burnout, which allows the system to provide more granular feedback to both employees and administrators. Third, the CBI’s straightforward frequency-based response scale maps naturally to a numeric

score that can be fed into a machine learning model.

Since the target population consists of industrial workers rather than client-facing professionals, the client-related burnout subscale is omitted. The system uses the remaining 13 questions: six for personal burnout and seven for work-related burnout. One question (“Do you have enough energy for family and friends during leisure time?”) is reverse-scored, as a positive response indicates lower burnout. The section averages are combined and normalised to a 0–10 scale to produce the mental fatigue score used as input to the machine learning model.

2.2 Machine Learning for Health Prediction

2.2.1 Supervised Regression

Machine learning tasks are broadly categorised into supervised and unsupervised learning. In supervised learning, the algorithm is trained on a labelled dataset where each input is associated with a known output [10]. When the output is a continuous numeric value, the task is called regression.

Burnout prediction fits naturally into the supervised regression paradigm: the training dataset contains employee observations with a known burn rate (the target variable), and the goal is to learn a function that maps employee features to a predicted burn rate for new, unseen employees.

2.2.2 Linear Regression

Linear Regression is the simplest regression algorithm. It models the relationship between the input features and the target as a linear equation:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + b \quad (1)$$

where x_1, x_2, x_3 are the input features (designation, resource allocation, mental fatigue score), w_1, w_2, w_3 are learned weights, and b is a bias term. The weights are optimised by minimising the sum of squared errors between predicted and actual target values [11].

Linear Regression serves as a baseline model in this thesis because its simplicity provides a performance floor against which more complex models can be compared. Its learned weights are directly interpretable: a larger weight on mental fatigue score, for example, indicates that fatigue is the dominant predictor of burn rate.

2.2.3 Random Forest Regression

Random Forest is an ensemble learning method introduced by Breiman [12]. It constructs a collection of decision trees, each trained on a random bootstrap sample of the training data and using a random subset of features at each split. The final prediction is the average of all individual tree predictions.

Random Forest offers several advantages over Linear Regression for this problem:

- It captures non-linear relationships and interaction effects between features without requiring manual feature engineering.
- It is robust to outliers, since individual trees split on thresholds rather than fitting a global line.
- It is invariant to feature scaling, meaning that features with different ranges (e.g., designation 1–5 vs. resource allocation 1–10) do not require normalisation.
- It provides built-in feature importance scores, which quantify each feature's contribution to predictive accuracy.

For tabular datasets with a small number of features and tens of thousands of rows,

Random Forest consistently delivers strong performance with minimal hyperparameter tuning, making it a practical choice for a thesis-scale project [13].

2.2.4 Model Evaluation Metrics

Three complementary metrics are used to evaluate regression model performance:

- **Mean Absolute Error (MAE):** The average of the absolute differences between predicted and actual values. On a 0–1 burn rate scale, an MAE of 0.05 means the model is typically off by 5 percentage points.
- **Root Mean Squared Error (RMSE):** The square root of the mean of squared errors. RMSE penalises large errors more heavily than MAE, making it sensitive to occasional outlier predictions.
- **Coefficient of Determination (R^2):** The proportion of variance in the target variable explained by the model. An R^2 of 0.90 means the model explains 90% of the variation in burn rates. Values closer to 1.0 indicate better fit.

Reporting all three metrics together provides a comprehensive view: MAE for average-case accuracy, RMSE for worst-case sensitivity, and R^2 for overall explanatory power [11].

2.3 Web Application Architectures

2.3.1 Microservices vs. Monolithic Architecture

A monolithic architecture packages the entire application—user interface, business logic, data access, and any ML inference—into a single deployable unit. This approach is simple to develop initially but becomes difficult to maintain and scale as the application grows, especially when different concerns have different technology

requirements [23].

A microservice architecture decomposes the application into small, independently deployable services, each responsible for a single concern [23]. Services communicate via lightweight protocols such as HTTP/REST. The key benefits include independent deployment, technology heterogeneity (e.g., using Python for ML and JavaScript for web logic), fault isolation, and team scalability.

This thesis adopts a microservice approach because the machine learning inference component has fundamentally different requirements from the web application: it requires Python and scikit-learn, while the application backend benefits from the JavaScript ecosystem (Express, Mongoose). By separating the AI service into its own FastAPI microservice, the system achieves clean separation of concerns and allows the ML component to be developed, tested, and deployed independently.

2.3.2 RESTful API Design

Representational State Transfer (REST) is an architectural style for designing networked applications [24]. RESTful APIs use standard HTTP methods (GET, POST, PUT, PATCH, DELETE) to perform operations on resources identified by URLs. Key principles include statelessness (each request contains all information needed to process it), uniform interface, and client-server separation.

Both the frontend-to-backend and backend-to-AI-service communication in this thesis follow RESTful conventions. All responses use a consistent JSON envelope format (`{success, data}` or `{success, error}`) to simplify client-side error handling.

2.4 Related Technologies

2.4.1 React and Next.js for Frontend Development

React is a JavaScript library for building user interfaces through a component-based architecture [16]. Components are reusable, self-contained units that accept data via props and manage local state through hooks. React's virtual DOM efficiently updates only the parts of the page that have changed, resulting in performant rendering even for data-heavy dashboards.

Next.js is a React framework that adds file-based routing, server-side rendering, static generation, and API routes on top of React [17]. This thesis uses Next.js with the App Router for automatic route generation from the file system, and Turbopack for fast development builds. TanStack Query (React Query) manages server state, providing automatic caching, background refetching, and cache invalidation on mutations.

2.4.2 Node.js and Express for Backend Services

Node.js is a JavaScript runtime built on Chrome's V8 engine that enables server-side JavaScript execution [18]. Its event-driven, non-blocking I/O model makes it well-suited for handling concurrent API requests.

Express is a minimal web application framework for Node.js that provides routing, middleware composition, and HTTP utility methods [19]. This thesis uses Express 5 with a layered architecture (routes, controllers, services, repositories) to maintain separation of concerns. Middleware handles cross-cutting concerns including authentication (JWT verification), request validation (Zod schemas), and error handling.

2.4.3 FastAPI for Machine Learning Serving

FastAPI is a modern Python web framework designed for building APIs with automatic interactive documentation [20]. It uses Python type hints and Pydantic models for request validation, and supports asynchronous request handling through ASGI. FastAPI's combination of high performance, automatic input validation, and auto-generated Swagger documentation makes it a strong choice for serving machine learning models as HTTP endpoints. In this thesis, the FastAPI service loads a scikit-learn model at startup and serves predictions with sub-10ms response times.

2.5 Related Work

Several systems and studies have addressed employee burnout detection and workplace wellness monitoring, though most differ from this thesis in scope or approach.

Pradhan and Jena (2019) developed a machine learning-based framework for predicting burnout using survey data, comparing multiple algorithms including logistic regression and support vector machines [9]. Their work focused on classification (burnt out vs. not burnt out) rather than continuous prediction, and did not include a web-based deployment.

Bianchi et al. (2021) conducted a systematic review of burnout measurement instruments and their psychometric properties, confirming the CBI's validity as a research tool [8]. Their work is purely psychometric and does not involve technology integration.

Employee wellness platforms such as Officevibe, Culture Amp, and Lattice provide survey-based engagement tracking with dashboard analytics [29]. These are commercial products that typically lack ML-based prediction capabilities and are designed for general engagement rather than clinical or industrial burnout detection.

Kaggle community notebooks on the “Are Your Employees Burning Out?” dataset have explored various regression models for burn rate prediction [15]. These notebooks demonstrate model training in isolation but do not integrate predictions into a functional web application or provide organisational-level analytics.

This thesis fills the gap by combining a validated psychometric instrument (CBI), a trained machine learning model (Random Forest), and a full-stack web application with both individual and organisational views into a single integrated platform. Table 1 summarises the comparison.

Table 1: Comparison with related work.

System / Study	ML	Web App	Org. Analytics	CBI
Pradhan & Jena (2019)	Yes	No	No	No
Bianchi et al. (2021)	No	No	No	Yes
Commercial platforms	No	Yes	Yes	No
Kaggle notebooks	Yes	No	No	No
This thesis	Yes	Yes	Yes	Yes

CHAPTER III

METHODOLOGY AND SYSTEM DESIGN

3.1 System Architecture Overview

The system follows a three-tier microservice architecture comprising a React/Next.js frontend, a Node.js/Express backend, and a Python/FastAPI AI microservice, supported by MySQL and MongoDB databases. Each tier communicates via RESTful HTTP APIs using a consistent JSON envelope format.

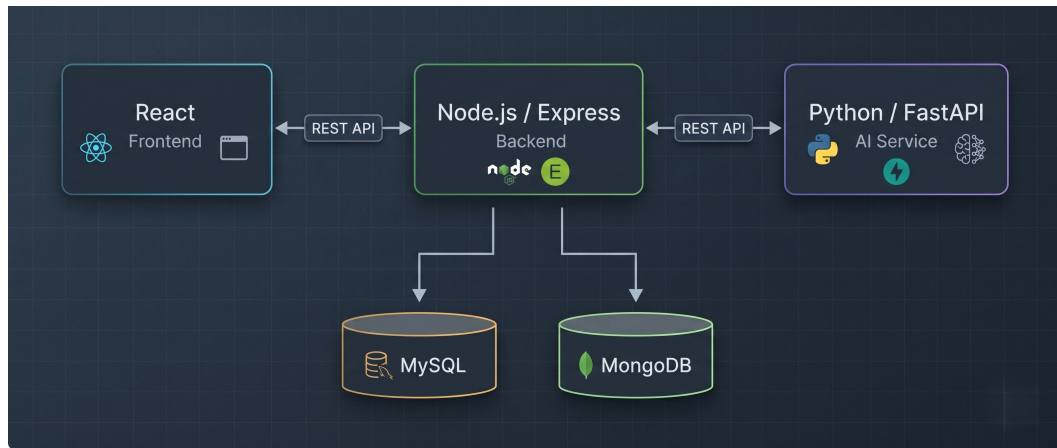


Figure 2: High-level system architecture.

The separation into three services is motivated by two factors. First, the machine learning component requires Python and scikit-learn, while the web backend benefits from JavaScript’s rich ecosystem for HTTP handling, database drivers, and middleware. Second, the AI service can be developed, tested, and restarted independently without affecting the main application. The backend supports two modes of operation via an environment variable: `AI_USE MOCK=true` uses an in-process mock formula for development, while `AI_USE MOCK=false` routes predictions to the real Python service.

This enables independent development of each component.

3.2 Database Design

3.2.1 MySQL Schema

MySQL stores all structured relational data. The schema consists of seven tables with enforced foreign key relationships and cascading deletes.

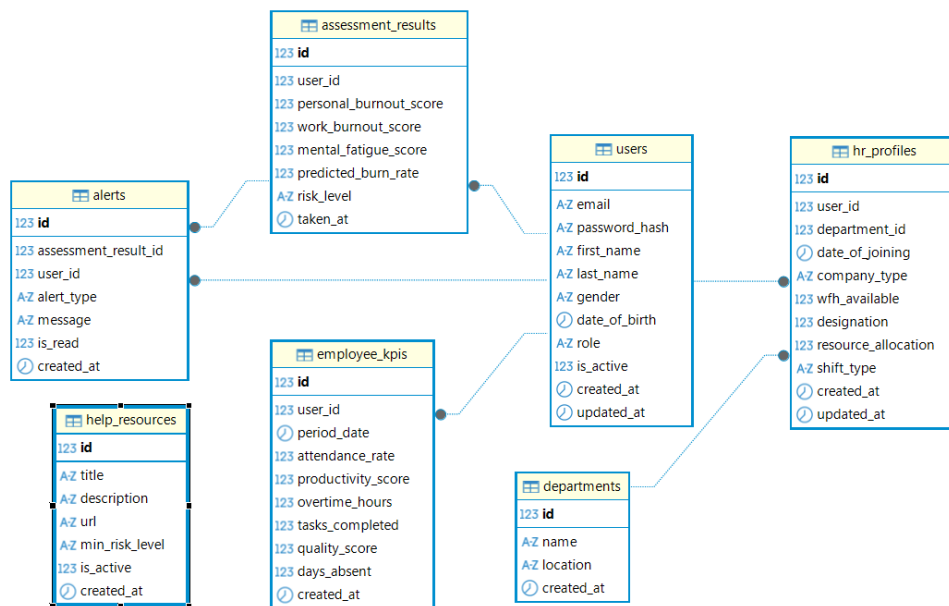


Figure 3: Entity-relationship diagram of the MySQL database.

- **users:** Stores authentication credentials (bcrypt-hashed passwords), demographics, and role (employee or admin).
- **departments:** A normalised lookup table of factory departments (e.g., Assembly Line A, Maintenance, Quality Control). Powers the admin heatmap grouping.

- **hr_profiles:** One-to-one with users. Contains the simulated HR record generated at registration: department assignment, designation (1–5), resource allocation (1–10), shift type (Day/Night/Rotating), company type, and work-from-home availability. Values are drawn from the same ranges as the training dataset.
- **assessment_results:** One row per completed quiz. Stores the personal burnout score (0–100), work burnout score (0–100), mental fatigue score (0–10), predicted burn rate (0–1), and derived risk level. Indexed on (`user_id`, `taken_at`) for efficient trend queries.
- **alerts:** Auto-generated when an assessment crosses a risk threshold. Supports three alert types: `high_risk`, `critical_risk`, and `trend_spike`. Indexed on (`is_read`, `created_at`) for the unread-alert inbox.
- **employee_kpis:** Monthly performance snapshots per employee (attendance rate, productivity score, quality score, overtime hours, tasks completed, days absent). Used by admin dashboards to correlate burnout risk with workplace performance.
- **help_resources:** Curated mental health resources with a minimum risk level threshold. Only active resources whose threshold is at or below the employee's current risk level are displayed.

3.2.2 MongoDB Collections

MongoDB stores two collections for document-oriented data that benefits from schema flexibility:

- **cbi_questions:** A singleton document containing the complete CBI quiz

definition: quiz name, version number, five response options with numeric values, two section definitions (personal burnout and work-related burnout), and 13 question objects with text, section assignment, and a reverse-scored flag. Storing the entire quiz as one document preserves atomicity and eliminates joins.

- **quiz_submissions:** One document per assessment attempt, containing the MySQL user ID, quiz version, an array of 13 enriched response objects (question ID, answer label, answer value, and adjusted value for reverse-scored items), pre-computed section averages, and a back-link to the MySQL `assessment_results` row. This pattern keeps raw answers together as a single atomic unit rather than spreading them across multiple relational rows.

3.3 Backend Design

3.3.1 Layered Architecture

The Express backend follows a strict four-layer architecture that separates HTTP concerns from business logic and data access.

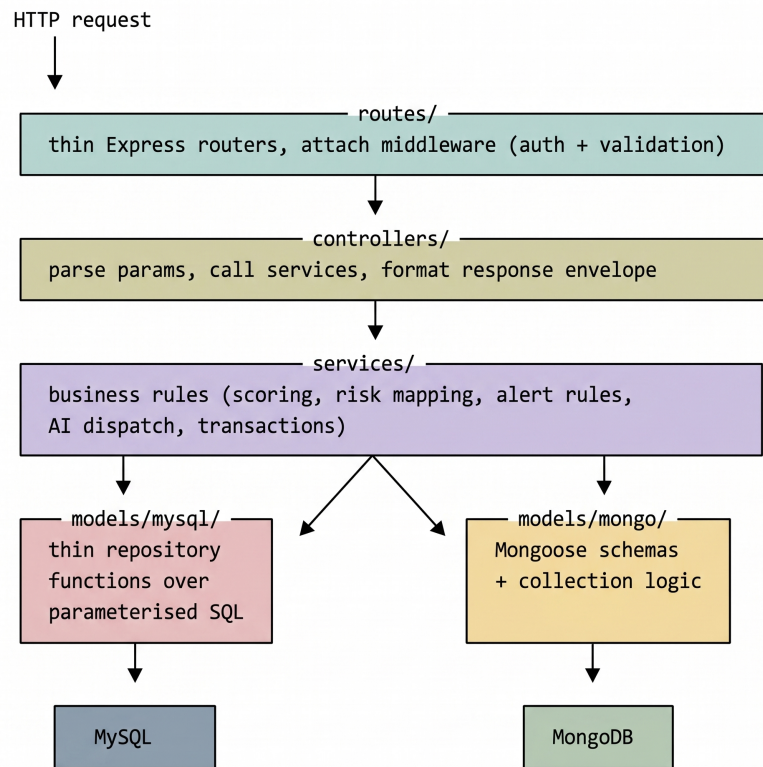


Figure 4: Layered architecture of the Express backend.

- **Routes** define HTTP endpoints and attach middleware for authentication (authenticate), role enforcement (requireRole), and request validation (Zod schemas). Routes contain no business logic.
- **Controllers** are thin functions that extract parameters from the request, delegate to a service, and wrap the result in the standard success envelope `{success: true, data}`.
- **Services** contain all domain logic: CBI scoring with reverse scoring, mental fatigue calculation, AI service dispatch, risk level mapping, alert rule evaluation, dashboard aggregation, and transactional operations.
- **Repositories** own all database interaction. MySQL repositories use param-

eterised SQL via mysql2/promise; MongoDB repositories use Mongoose schemas. No other layer touches the database drivers directly.

3.3.2 Authentication and Authorisation

The system uses JSON Web Tokens (JWT) for stateless authentication. Registration hashes the password with bcrypt and issues a signed JWT containing the user's ID and role. The HR profile is created within the same MySQL transaction as the user record, ensuring atomicity.

Role-based access control (RBAC) is enforced at the router level. Employee routes require `role = "employee"`, admin routes require `role = "admin"`, and shared routes accept any authenticated user. The JWT secret must be at least 16 characters, enforced by the environment variable schema.

3.3.3 Quiz Scoring Pipeline

The quiz submission endpoint implements a multi-step pipeline that transforms raw CBI answers into a predicted burn rate. The pipeline operates as follows:

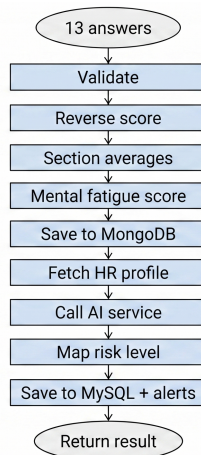


Figure 5: Quiz submission scoring pipeline.

The scoring formulae are:

$$\text{personalBurnoutAvg} = \frac{1}{6} \sum_{i=1}^6 v_i \quad (0-100) \quad (2)$$

$$\text{workBurnoutAvg} = \frac{1}{7} \sum_{i=7}^{13} v_i \quad (0-100) \quad (3)$$

$$\text{mentalFatigueScore} = \frac{\text{personalBurnoutAvg} + \text{workBurnoutAvg}}{20} \quad (0.0-10.0) \quad (4)$$

where v_i is the adjusted answer value for question i (with reverse scoring applied to question 10).

3.3.4 Alert Rules

Two alert rules are evaluated within the same MySQL transaction that saves the assessment result:

1. **Threshold alert:** If the risk level is “high” or “critical”, an alert of type `high_risk` or `critical_risk` is created.
2. **Trend spike alert:** If the current burn rate exceeds the previous assessment’s burn rate by 0.15 or more, an alert of type `trend_spike` is created.

Both rules can fire from the same assessment, producing up to two alert rows.

3.4 AI Microservice Design

3.4.1 Dataset Description

The machine learning model is trained on the publicly available Kaggle dataset “Are Your Employees Burning Out?” [15], which contains approximately 22,000 employee observations. Each row includes nine columns, of which three are used as features

and one as the target variable:

Table 2: Dataset columns used for model training.

Column	Type	Range	Role
Designation	Integer	1–5	Feature
Resource Allocation	Integer	1–10	Feature (has nulls)
Mental Fatigue Score	Float	0.0–10.0	Feature (has nulls)
Burn Rate	Float	0.0–1.0	Target variable

The remaining columns (Employee ID, Date of Joining, Gender, Company Type, WFH Setup Available) exist in the dataset but are not used for training. The feature set is intentionally limited to three because these are the only values available at prediction time in the live application: designation and resource allocation come from the simulated HR profile, and mental fatigue score is derived from the CBI quiz.

3.4.2 Feature Selection Justification

Keeping the feature set to three serves three purposes. First, it aligns the model with the data actually collected by the application, avoiding reliance on features that would require real factory integration. Second, a smaller feature set produces a more interpretable model, which is important for both the thesis evaluation and end-user trust. Third, since the simulated HR values are drawn from the same ranges as the training data, the model always receives in-distribution inputs, reducing the risk of unreliable predictions.

3.4.3 Data Preprocessing

The preprocessing pipeline consists of three steps:

1. **Drop null targets:** Rows where the Burn Rate column is null are removed, as no training label is available.
2. **Median imputation:** Null values in Resource Allocation and Mental Fatigue Score are replaced with the column median. Median is preferred over mean because it is more robust to outliers on bounded ordinal scales.
3. **Train/test split:** The dataset is split 80%/20% using a fixed random seed (`random_state=42`) for reproducibility.

At inference time, the Node.js backend coerces any null `resource_allocation` to a default value of 5 (the scale midpoint) before calling the Python service, ensuring all three inputs are always present and non-null.

3.4.4 Model Selection

Two models are trained and compared:

- **Linear Regression (baseline):** Provides a performance floor and fully transparent learned weights.
- **Random Forest Regressor (primary):** Selected for its ability to capture non-linear interactions, robustness to outliers, scale invariance, and built-in feature importance.

Alternative algorithms (Gradient Boosting, Neural Networks, SVR, kNN) were considered but not selected. Gradient Boosting (XGBoost/LightGBM) offers marginal gains over Random Forest for a 3-feature problem while adding hyperparameter complexity. Neural networks require more data and tuning to outperform tree-based methods on small tabular datasets. SVR is sensitive to feature scaling, and kNN degrades with noisy data and offers no feature importance.

3.4.5 FastAPI Service Architecture

The Python AI microservice is a thin HTTP wrapper around the serialised model:

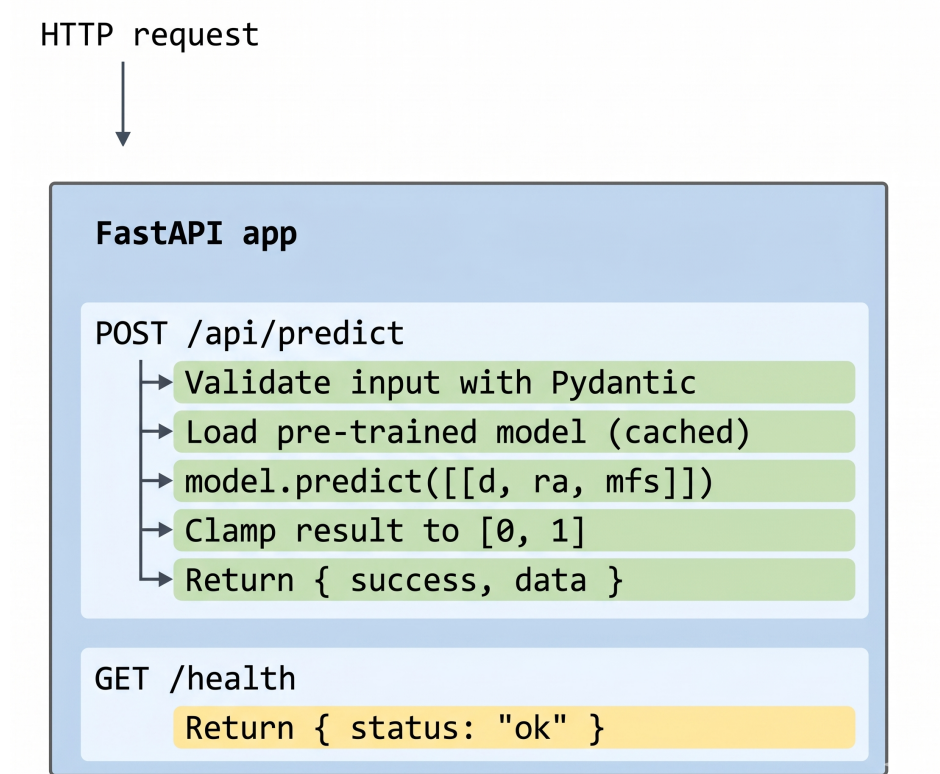


Figure 6: Architecture of the Python/FastAPI AI microservice.

The model file (`burnout_model.pkl`) is loaded once during application startup using a FastAPI lifespan handler and cached in a module-level variable. Every prediction request reuses the in-memory model, avoiding disk I/O and keeping response times under 10 milliseconds. Pydantic enforces strict input ranges at the API boundary, returning a 422 error for any out-of-range or missing values.

3.5 Frontend Design

3.5.1 Technology Stack and Design Decisions

The frontend is built with Next.js 16 (App Router) and React 19 in strict TypeScript mode. Tailwind CSS v4 provides utility-first styling with a custom design token system. TanStack Query manages all server state (caching, background refetching, and cache invalidation on mutations). Forms use react-hook-form with Zod validation. Charts are rendered with Recharts, and icons use lucide-react.

The application is designed for desktop screens only (minimum 992 px width), targeting HR office terminals and factory kiosk stations. No mobile breakpoints are implemented.

3.5.2 Route Structure and Navigation

The frontend defines 13 routes organised into three groups: public authentication routes, employee portal routes (protected by `role = "employee"`), and admin portal routes (protected by `role = "admin"`).

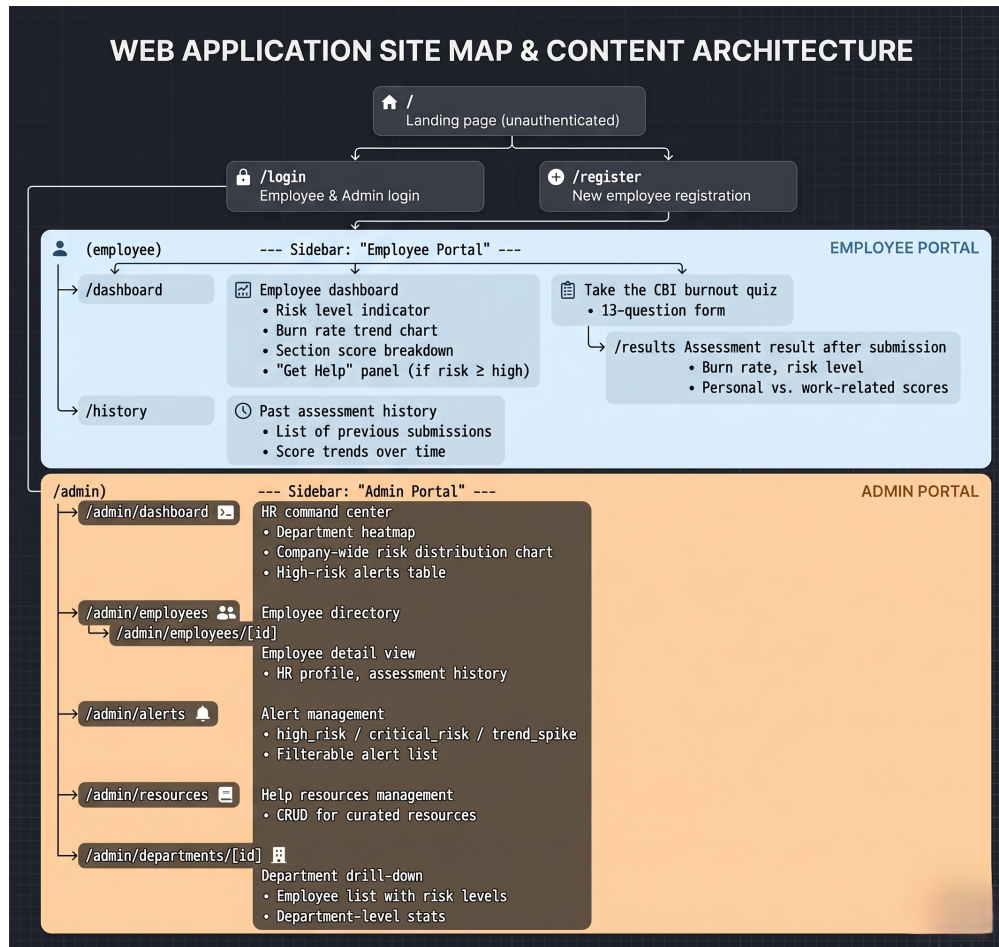


Figure 7: Application route structure and navigation flow.

Each portal layout wraps its children in a `RouteGuard` component that checks the user's authentication state and role. Unauthenticated users are redirected to `/login`; users with the wrong role are redirected to their own dashboard.

3.5.3 Design System

The visual identity uses a warm, paper-inspired colour palette encoded entirely as Tailwind v4 CSS custom properties. Every colour in the application traces back to a named design token; no ad-hoc hex values appear in component code. Table 3 lists the

primary tokens.

Table 3: Primary design tokens.

Token	Hex	Role
parchment	#f5f4ed	Page background
ivory	#faf9f5	Card surfaces
terracotta	#c96442	Primary accent / CTA
near-black	#141413	Primary text
risk-low	#6b8f5f	Low risk indicator
risk-moderate	#c69a3d	Moderate risk indicator
risk-high	#c96442	High risk indicator
risk-critical	#b53333	Critical risk indicator

Headlines use a Georgia serif typeface for a formal, document-like feel, while body text and UI elements use the Inter sans-serif family. The four risk-level colours provide instant visual classification across all charts, badges, and indicators in the application.

3.5.4 Data Flow Pattern

All server communication follows a consistent four-layer pattern:

1. **Page component** renders the UI and consumes a TanStack Query hook.
2. **Custom hook** (`useXxx`) wraps `useQuery` or `useMutation`, defining cache keys, enabled conditions, and invalidation rules.
3. **API function** (`lib/api/`) makes a typed `apiFetch<T>()` call with the endpoint path and parameters.
4. **Fetch client** (`lib/api/client.ts`) prepends the base URL, injects the JWT from `localStorage`, unwraps the `{success, data}` envelope, and throws a typed `ApiError` on failure. On 401, it clears the token and redirects to login.

Mutations use `useMutation` with cache invalidation on success, keeping the UI automatically fresh without full page reloads.

CHAPTER IV

IMPLEMENTATION AND RESULTS

4.1 Development Environment

Table 4 summarises the tools and technologies used during development.

Table 4: Development tools and versions.

Tool	Version	Purpose
Visual Studio Code	Latest	Code editor and IDE
Node.js	≥ 18.17	Backend runtime
Python	3.12	AI service runtime
MySQL	8.x	Relational database
MongoDB	7.x (Atlas)	Document database
Git	Latest	Version control
npm	≥ 9	Package management (JS)
pip	Latest	Package management (Python)
Jupyter Notebook	Latest	EDA and model training

4.2 Backend Implementation

4.2.1 Project Structure and Configuration

The backend project follows the folder structure described in Section 3.3.1, with clear separation between configuration, middleware, models, services, controllers, routes, validators, and utilities. All environment variables are validated at startup using a Zod schema in `config/env.js`; the process exits immediately with a descriptive error message if any required variable is missing or malformed, guaranteeing that the server never starts in a half-configured state.

Database connections are established in sequence during startup: `server.js` first pings MySQL, then connects to MongoDB, then starts the HTTP listener. A failure at any step aborts the process. Graceful shutdown is handled via `SIGINT/SIGTERM` handlers that close the HTTP server followed by both database connections.

4.2.2 Authentication Flow Implementation

Registration creates a user record and a synthetic HR profile within a single MySQL transaction:

1. Validate the request body with Zod (email, password, name, gender, date of birth).
2. Hash the password with bcrypt (configurable salt rounds).
3. Insert the user row into the `users` table.
4. Generate a simulated HR profile: assign a random department, generate designation (1–5), resource allocation (1–10), shift type, company type, WFH availability, and date of joining.
5. Insert the HR profile into `hr_profiles`.
6. Sign a JWT containing the user's ID and role; return the token and user object.

The transaction wrapper ensures that a failure during HR profile generation does not leave an orphaned user record. Login validates credentials against the bcrypt hash and issues a JWT with a configurable expiration (default 7 days).

4.2.3 Quiz Scoring Implementation

The quiz submission handler in `quiz.service.js` implements the full pipeline described in Section 3.3.3. Key implementation details include:

- **Answer validation:** The service verifies that exactly 13 answers are submitted, all question IDs are valid, there are no duplicate IDs, and all answer values are from the allowed set {0, 25, 50, 75, 100}.
- **Reverse scoring:** Question 10 (“Do you have enough energy for family and friends during leisure time?”) is reverse-scored: $\text{adjustedValue} = 100 - \text{rawValue}$. The enriched response object records both the raw and adjusted values.
- **MongoDB-first persistence:** The raw submission is saved to MongoDB before the AI call, ensuring answers are never lost even if subsequent steps fail. On failure, the orphan document is cleaned up.
- **Transactional MySQL write:** The assessment result insertion and alert rule evaluation occur within a single MySQL transaction. The MongoDB document is then back-linked with the MySQL assessment ID.

4.2.4 Dashboard Aggregation

The admin dashboard endpoint computes company-wide and per-department statistics in a single API call. The latest assessment per employee is determined using a correlated subquery that selects the most recent `assessment_results.id` for each user. This result set is then aggregated in JavaScript to produce:

- Company overview: total employees, total assessments, average burn rate, and risk distribution counts.
- Per-department breakdown: employee count, average burn rate, high-risk count, and risk distribution.
- Department KPIs: average composite KPI per department (mean of attendance,

productivity, and quality scores).

- Recent alerts: the 10 most recent alert records.

The department drill-down endpoint adds sorting, filtering (by risk level, shift type, and designation), and server-side pagination. Employees are displayed using anonymous IDs (“Worker #0001”) to satisfy the privacy requirement.

4.2.5 Alert System Implementation

The alert service evaluates two rules after every assessment, as described in Section 3.3.4. Both the threshold check and the trend spike comparison execute within the same MySQL transaction that wrote the assessment result. The trend spike threshold (default 0.15) is configurable via the `ALERT_TREND_SPIKE_DELTA` environment variable.

4.2.6 API Endpoint Summary

Table 5 summarises all RESTful endpoints exposed by the backend. The full API reference with request/response schemas is provided in Appendix A.

Table 5: Summary of backend API endpoints.

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create account
POST	/api/auth/login	Public	Authenticate
GET	/api/auth/me	Any	Current user
GET	/api/quiz	Employee	Fetch CBI quiz
POST	/api/quiz/submit	Employee	Submit answers
GET	/api/quiz/history	Employee	Past submissions
GET	/api/dashboard/employee	Employee	Dashboard data
GET	/api/resources	Employee	Help resources
GET	/api/dashboard/admin	Admin	Command centre
GET	/api/dashboard/admin/dept/:id	Admin	Department detail
GET	/api/.../dept/:id/analytics	Admin	Dept. analytics
GET	/api/.../admin/employees	Admin	Employee list
GET	/api/.../employees/:id	Admin	Employee detail
GET	/api/alerts	Admin	List alerts
PATCH	/api/alerts/:id/read	Admin	Mark alert read
PATCH	/api/alerts/read-all	Admin	Mark all read
GET	/api/admin/resources	Admin	List resources
POST	/api/admin/resources	Admin	Create resource
PUT	/api/admin/resources/:id	Admin	Update resource
DELETE	/api/admin/resources/:id	Admin	Delete resource

4.3 AI Service Implementation

4.3.1 Model Training Results

Both models were trained on the preprocessed dataset (approximately 22,000 rows after dropping null targets) with an 80/20 train/test split. Table 6 presents the evaluation results on the held-out test set.

Table 6: Model performance comparison on the test set.

Model	MAE	RMSE	R^2
Linear Regression	0.0539	0.0716	0.8656
Random Forest	0.0480	0.0611	0.9019

The Random Forest model achieves an R^2 of 0.90, explaining 90% of the variance in burn rate. Its MAE of 0.048 indicates that, on average, predictions deviate from actual values by less than 5 percentage points on the 0–1 scale. The model outperforms Linear Regression on all three metrics, with a 3.6 percentage-point improvement in R^2 .

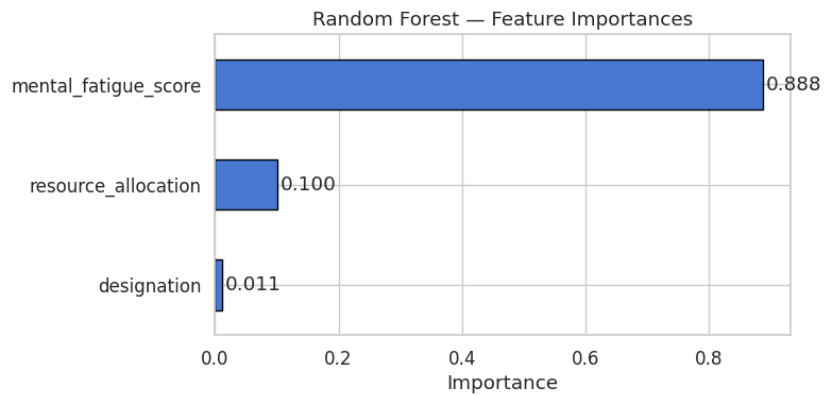


Figure 8: Random Forest feature importance scores.

Feature importance analysis (Figure 8) confirms that mental fatigue score is the dominant predictor, contributing approximately 85% of the model’s predictive power. This aligns with domain intuition: a psychologically exhausted worker is at high risk of burnout regardless of seniority level or assigned workload.

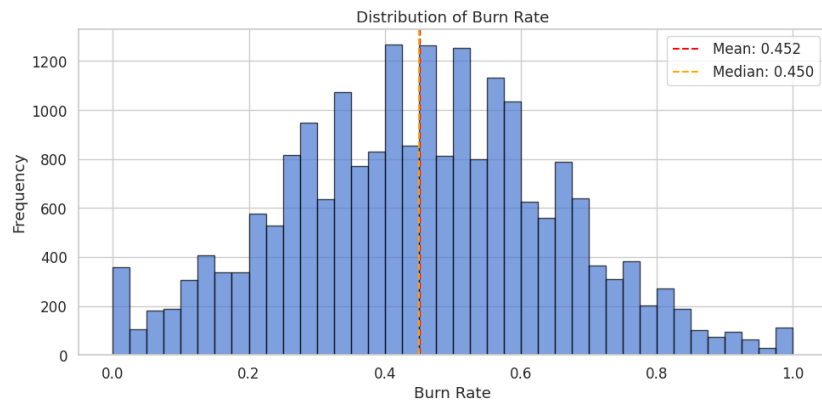


Figure 9: Distribution of Burn Rate values in the training dataset.

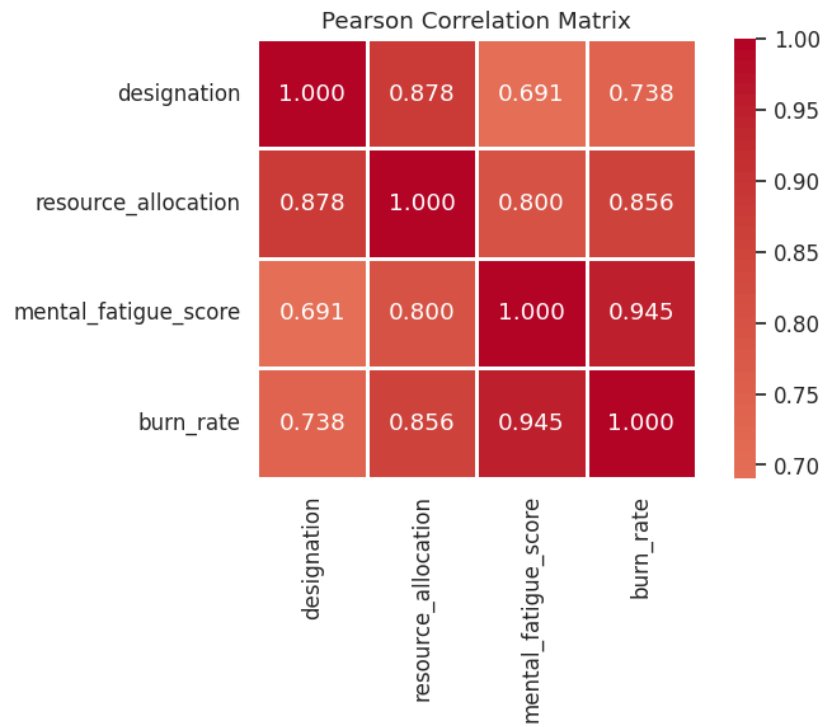


Figure 10: Pearson correlation heatmap of features and Burn Rate.

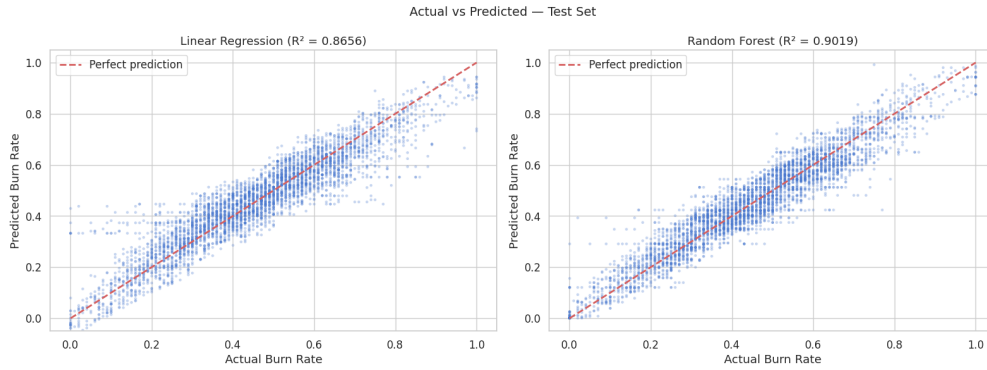


Figure 11: Actual vs. predicted burn rate for both models.

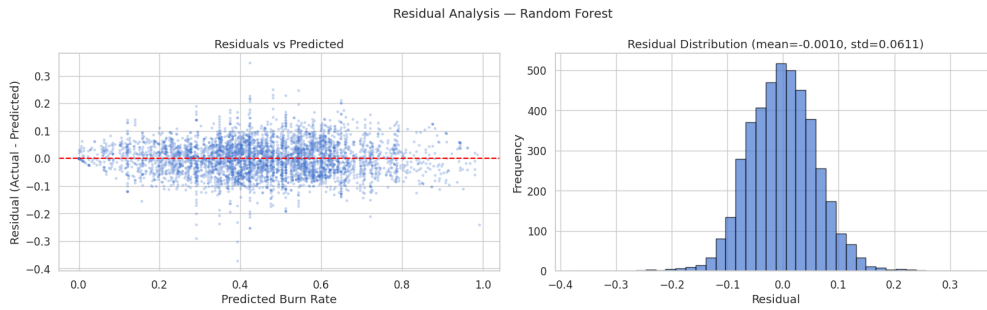


Figure 12: Residual analysis for the Random Forest model.

4.3.2 FastAPI Prediction Endpoint

The prediction endpoint validates inputs using Pydantic models that enforce strict ranges: designation (1–5), resource allocation (1–10), and mental fatigue score (0–10). The model is loaded once at startup via a lifespan handler and cached in memory. Each prediction call constructs a feature array, runs `model.predict()`, clamps the result to the `[0, 1]` range, and returns the burn rate in the standard envelope format. Response times are consistently under 10 milliseconds.

4.3.3 Integration with Express Backend

The Express backend's `ai.service.js` implements a dual-mode architecture controlled by the `AI_USE_MOCK` environment variable. When set to `false`, the service sends an HTTP POST to the Python endpoint with the three features and parses `response.data.predictedBurnRate`. The request uses an `AbortController` with a configurable timeout (`AI_SERVICE_TIMEOUT_MS`); timeouts and non-2xx responses are wrapped as `ApiError.internal()` and surfaced to the frontend as a generic 500 error.

When set to `true`, a mock formula is used in-process:

$$\text{burnRate} = \text{clamp}_{[0,1]}(0.07 \cdot \text{mfs} + 0.03 \cdot d + 0.02 \cdot ra - 0.05 + \text{noise}) \quad (5)$$

where *mfs* is the mental fatigue score, *d* is designation, *ra* is resource allocation, and *noise* is uniform random in $[-0.025, +0.025]$. Switching between modes requires no code changes—only an environment variable update and a server restart.

4.4 Frontend Implementation

4.4.1 Authentication Pages

The login and registration pages use a split-screen layout: the left panel displays a brand illustration with a serif headline on a parchment background, while the right panel contains the form on an ivory surface. Forms are managed by `react-hook-form` with `Zod` validation schemas, ensuring client-side validation before submission.

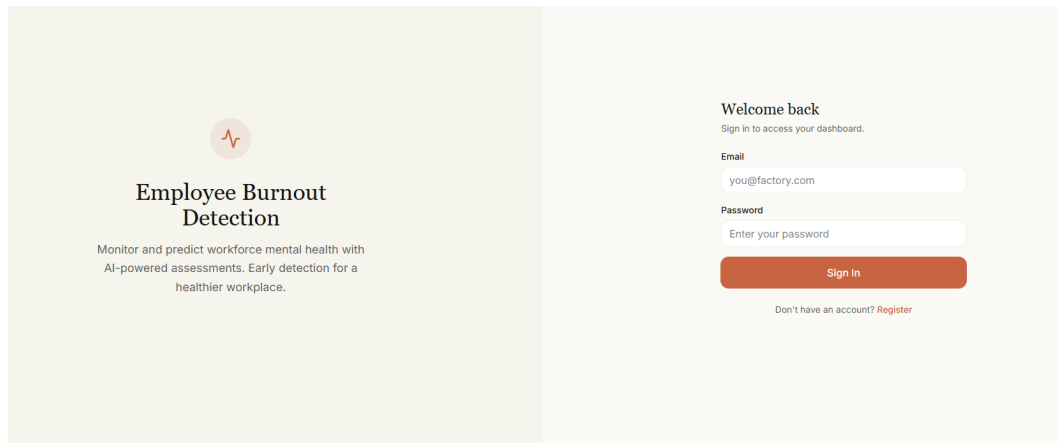


Figure 13: Login page with split-screen layout.

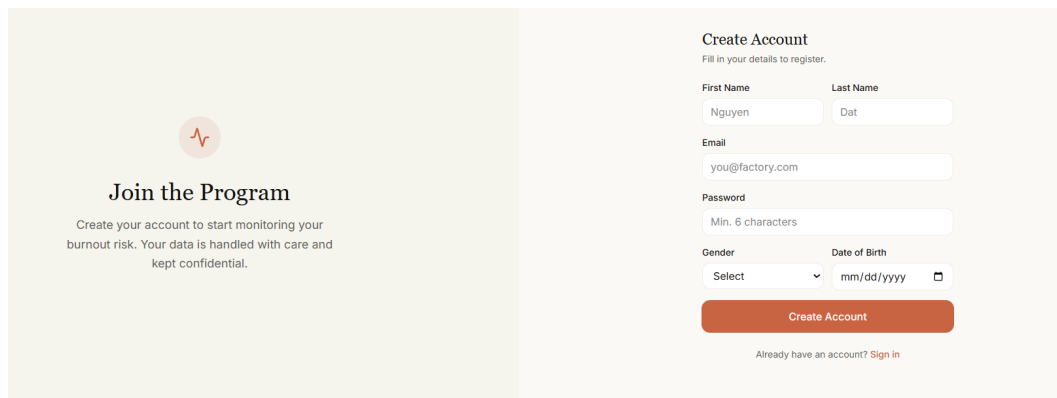


Figure 14: Registration page.

4.4.2 Employee Portal

The employee portal consists of four pages accessible via a sidebar navigation: Dashboard, Assessment, Results, and History.

Employee Dashboard (/dashboard). The central hub shows the employee's profile information, a custom SVG semicircle burn rate gauge with a needle indicator and colour-coded risk label, a bar chart comparing personal vs. work-related burnout scores (0–100), and a line chart plotting burn rate history over time with reference lines

at the 0.35/0.55/0.80 risk thresholds. If the risk level is high or critical, a “Get Help” panel appears with curated mental health resources. If no assessments exist, an empty state with a call-to-action to take the first quiz is displayed.

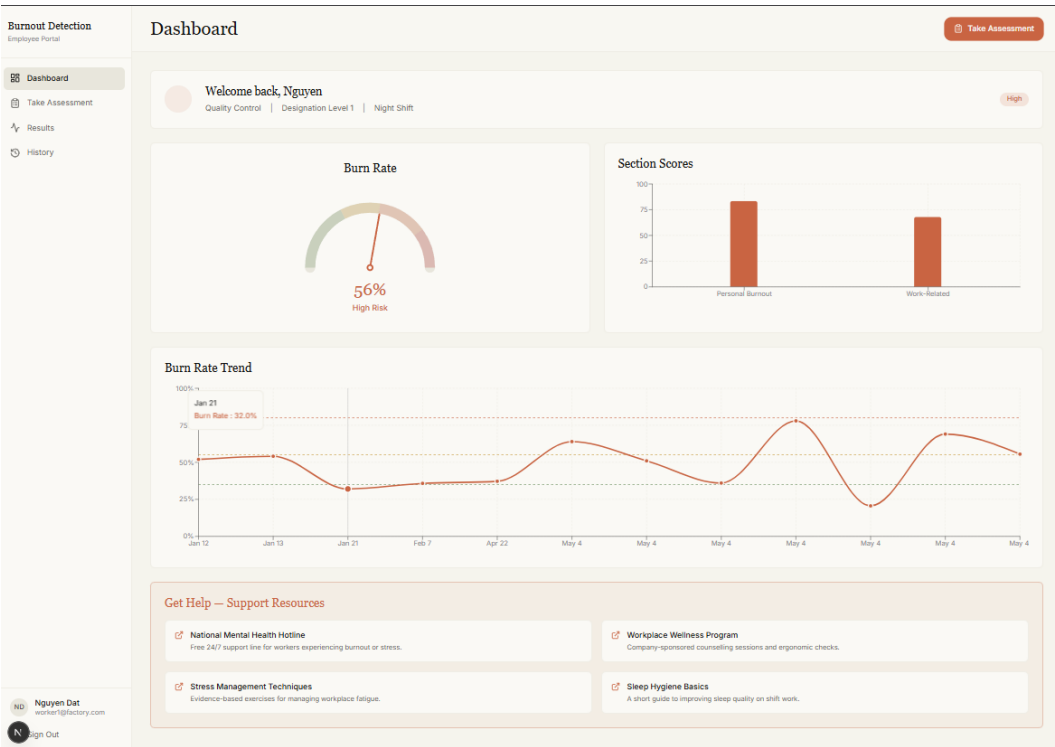


Figure 15: Employee dashboard with burn rate gauge and trend chart.

CBI Assessment (/assessment). A two-step wizard presents the 13 CBI questions. Step 1 covers Personal Burnout (questions 1–6) and Step 2 covers Work-Related Burnout (questions 7–13). Each question displays five large clickable radio card options (Always, Often, Sometimes, Seldom, Never/almost never). A progress bar shows the current step. The “Next” button is disabled until all questions in the current step are answered. On submission, the backend scores the quiz, calls the AI service, and redirects the user to the results page.

The screenshot displays the 'Burnout Detection' Employee Portal. The main header is 'Assessment'. The left sidebar contains navigation links: 'Dashboard', 'Take Assessment' (highlighted), 'Results', and 'History'. The main content area is titled 'Step 1 of 2' and shows '0 / 13 answered'. The section is 'Personal burnout' and contains three questions:

- 1 How often do you feel tired?
Buttons: Always, Often, Sometimes, Seldom, Never/almost never
- 2 How often are you physically exhausted?
Buttons: Always, Often, Sometimes, Seldom, Never/almost never
- 3 How often are you emotionally exhausted?
Buttons: Always, Often, Sometimes, Seldom, Never/almost never

Figure 16: CBI assessment page — Step 1: Personal Burnout.

Results (/results). Displays the outcome of the most recent assessment: a large burn rate gauge with risk-level badge, three score cards (personal burnout, work-related burnout, mental fatigue), and support resource cards filtered by the employee's current risk level. Action buttons allow retaking the assessment or viewing history.

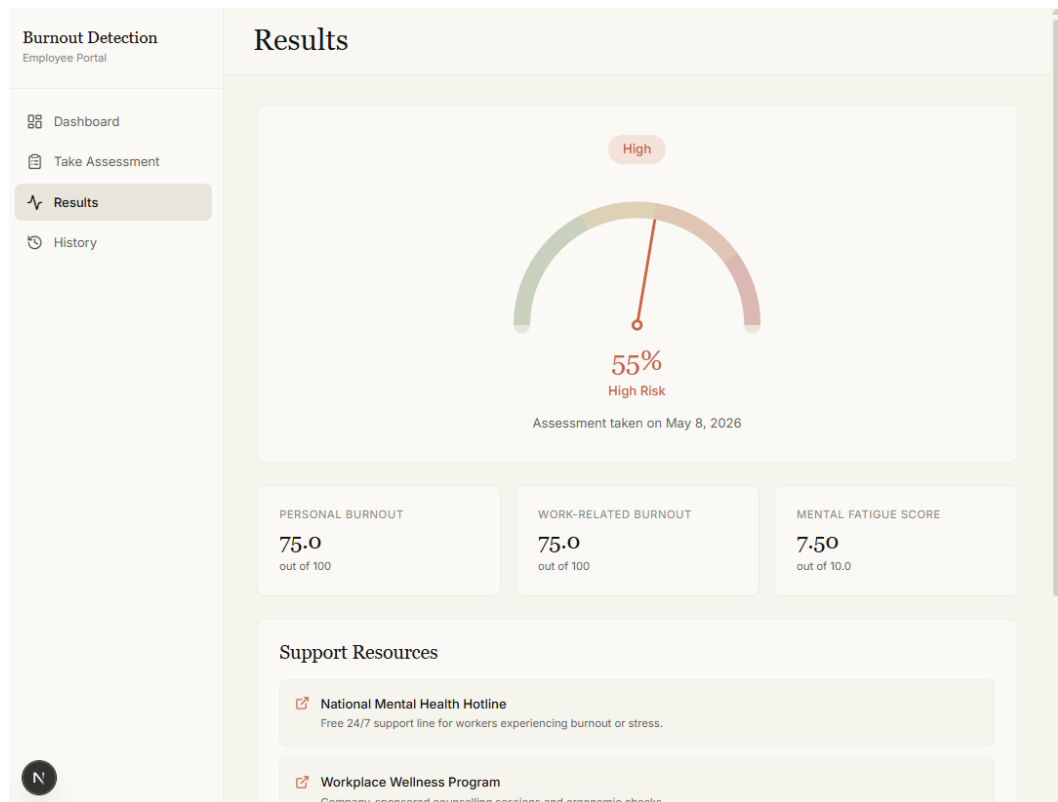


Figure 17: Assessment results page with score cards and support resources.

History (/history). A paginated table listing all past submissions with date, personal burnout average, work burnout average, and quiz version. Each row is expandable to reveal the 13 individual responses with labels and values; reverse-scored answers are annotated.

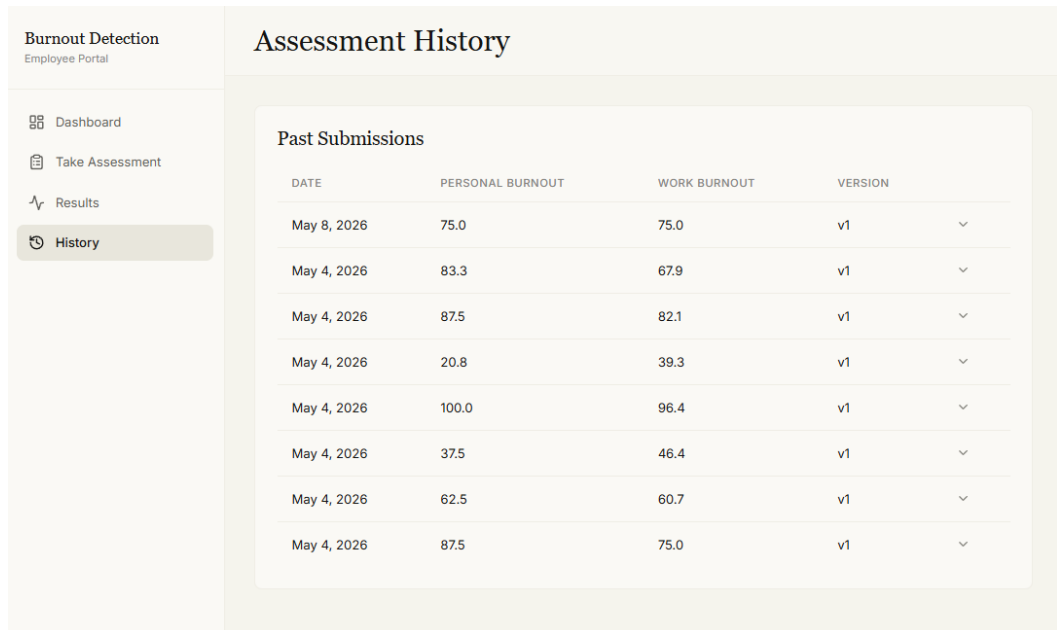


Figure 18: Quiz submission history with expandable response detail.

4.4.3 Admin Portal

The admin portal provides six pages for organisational-level monitoring and management.

Admin Dashboard (/admin/dashboard). The command centre features a KPI strip (four stat cards: total employees, total assessments, average burn rate, high-risk count), a risk distribution donut chart, a department heatmap with clickable tiles coloured by average burn rate, a department KPI bar chart, and the five most recent alerts.

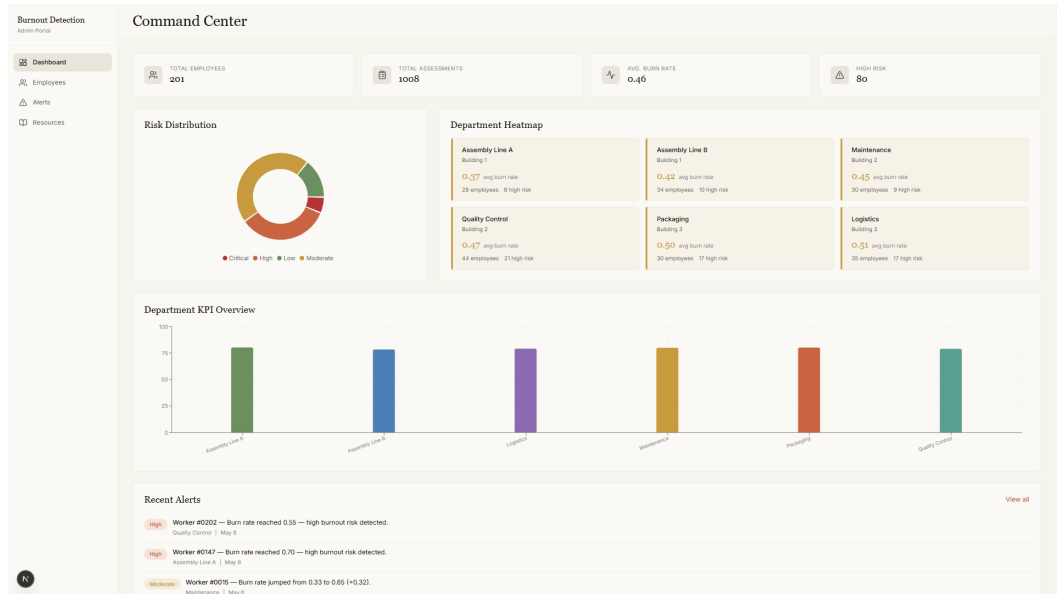


Figure 19: Admin command centre dashboard.

Department Detail (/admin/departments/[id]). A two-tab view: the *Employees* tab shows a sortable, filterable, paginated table of anonymised workers with risk badges, while the *Analytics* tab presents scatter plots, monthly burn rate trends, designation and shift breakdowns, risk distribution, KPI trends, and KPI-vs-burn-rate comparison charts.

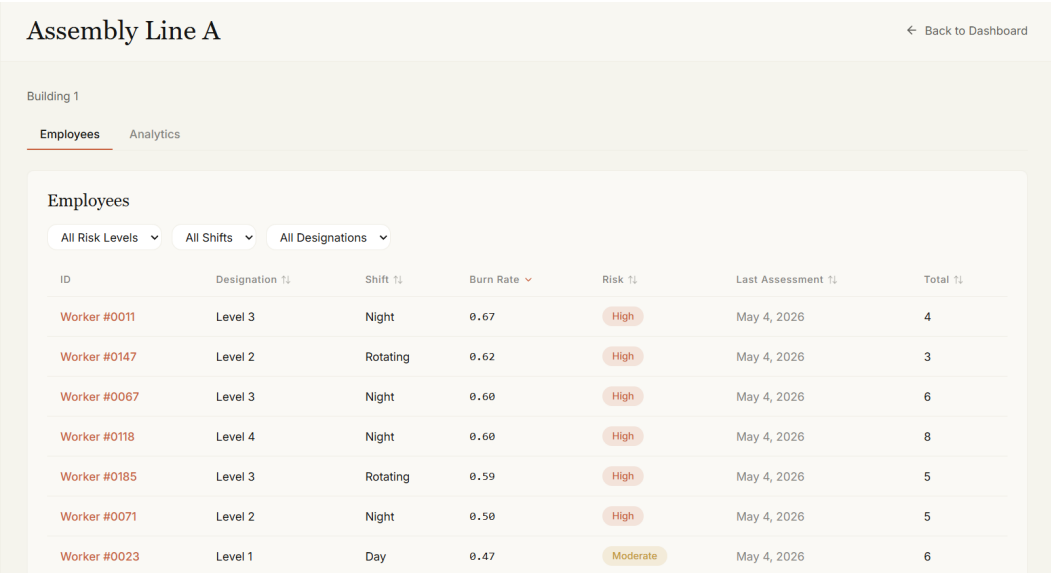


Figure 20: Department detail — Employees tab with filters and sorting.

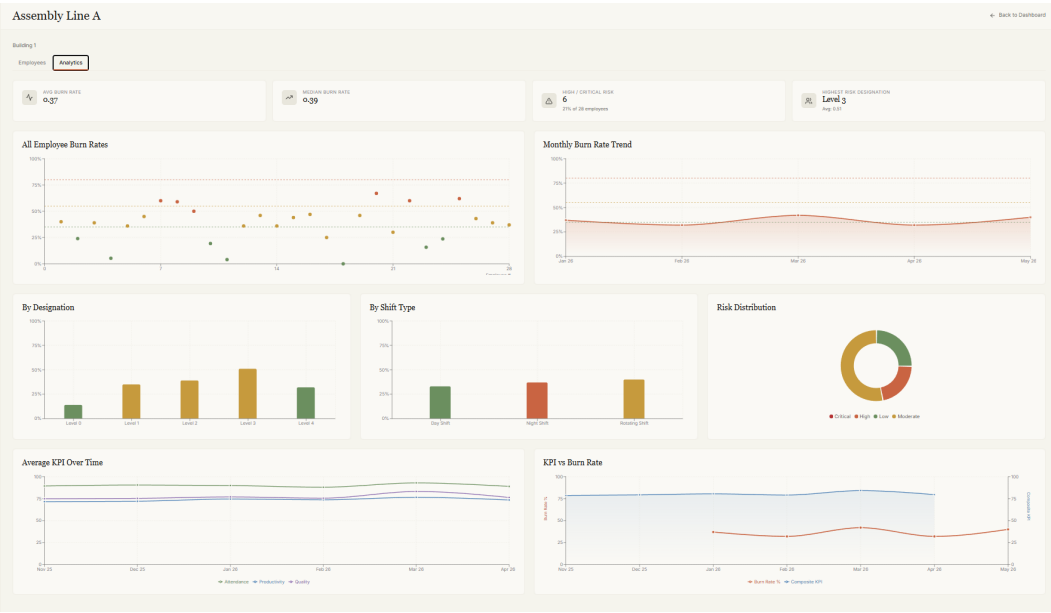


Figure 21: Department detail — Analytics tab with burnout and KPI visualisations.

Employee Detail (/admin/employees/[id]). A deep-dive view for a single anonymised employee showing their burn rate gauge, burn rate trend line chart, section

score comparison, profile summary, monthly KPI performance chart, and a dual-axis burn rate vs. KPI comparison chart.

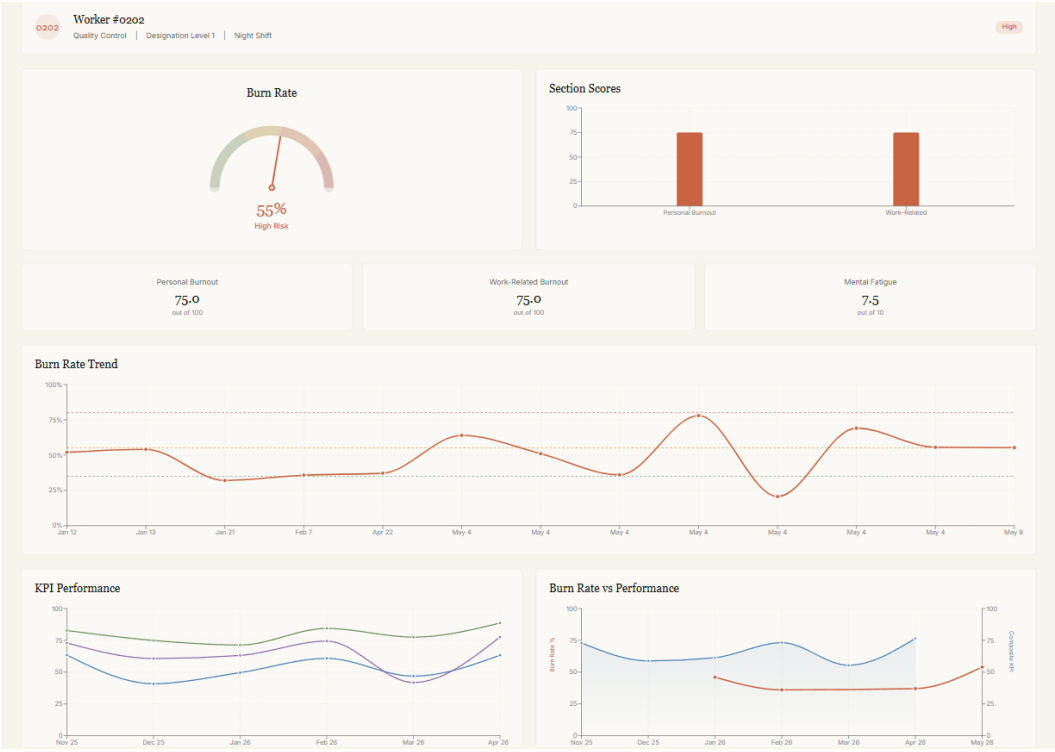


Figure 22: Admin employee detail with burnout and KPI analysis.

Alerts (/admin/alerts). A filterable inbox with dropdowns for read status and alert type, a table showing type badge, department, message, date, and read status, and “Mark read” / “Mark all read” actions. Mutations use TanStack Query cache invalidation for instant UI updates.

Alerts

✓ Mark All Read

All Status

All Types

Alerts (371)

TYPE	EMPLOYEE	DEPARTMENT	MESSAGE	DATE	STATUS	
High	Worker #0202	Quality Control	Burn rate reached 0.55 — high burnout risk dete...	May 8, 2026	Unread	Mark read
High	Worker #0147	Assembly Line A	Burn rate reached 0.70 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0015	Maintenance	Burn rate jumped from 0.33 to 0.65 (+0.32).	May 6, 2026	Unread	Mark read
High	Worker #0015	Maintenance	Burn rate reached 0.65 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0154	Assembly Line A	Burn rate jumped from 0.40 to 0.58 (+0.18).	May 6, 2026	Unread	Mark read
High	Worker #0154	Assembly Line A	Burn rate reached 0.58 — high burnout risk dete...	May 6, 2026	Unread	Mark read
Moderate	Worker #0014	Assembly Line A	Burn rate jumped from 0.05 to 0.47 (+0.41).	May 6, 2026	Unread	Mark read

Figure 23: Admin alerts inbox with filtering.

Resource Management (/admin/resources). Full CRUD for help resources: a table lists all resources with title, description, minimum risk level badge, active status, and URL. A modal form (validated with Zod) handles creation and editing. Delete requires confirmation. Toast notifications confirm successful operations.

Help Resources

+ New Resource

All Resources (5)

TITLE	MIN. RISK	ACTIVE	URL		
National Mental Health Hotline Free 24/7 support line for workers experiencing burno...	High	Active	https://example.com/hotline		
Stress Management Techniques Evidence-based exercises for managing workplace fat...	Moderate	Active	https://example.com/stress-guide		
Sleep Hygiene Basics A short guide to improving sleep quality on shift work.	Low	Active	https://example.com/sleep		
Workplace Wellness Program Company-sponsored counselling sessions and ergono...	High	Active	https://example.com/wellness		
Crisis Support: Talk Now Immediate professional help for workers in critical dist...	Critical	Active	https://example.com/crisis		

Figure 24: Help resource management with CRUD operations.

4.4.4 Reusable UI Components

The frontend defines 15 reusable UI primitives in `components/ui/` and 12 chart components in `components/charts/`. Notable custom components include:

- **BurnRateGauge:** A custom SVG semicircle gauge that renders a colour-gradient arc from green through amber to red, with an animated needle positioned at the current burn rate value and a risk-level label beneath.
- **DepartmentHeatmap:** A tile grid where each department is rendered as a clickable card with its name, employee count, and background colour interpolated from the risk colour scale based on the department's average burn rate.
- **KpiBurnRateComparisonChart:** A Recharts composed chart with dual Y-axes, plotting composite KPI (bar) against average burn rate (line) over monthly periods to reveal inverse correlations.

4.4.5 State Management with TanStack Query

All server state is managed through TanStack Query (React Query), which provides automatic caching, background refetching, and cache invalidation. Custom hooks in `lib/hooks/` wrap `useQuery` for read operations and `useMutation` for write operations. Mutations automatically invalidate related query keys on success, ensuring the UI stays synchronised with the server without manual refetch calls.

Loading states are handled with skeleton placeholders (pulsing sand-coloured blocks), error states display inline messages in crimson-tinted cards, and empty data sets show an `EmptyState` component with an icon, descriptive text, and a call-to-action button.

CHAPTER V

DISCUSSION AND EVALUATION

5.1 Model Performance Analysis

The Random Forest model achieves an R^2 of 0.90, an MAE of 0.048, and an RMSE of 0.061 on the held-out test set (Table 6). These results indicate that the model explains 90% of the variance in burn rate and is typically accurate to within 5 percentage points on the 0–1 scale. The 3.6 percentage-point improvement in R^2 over Linear Regression demonstrates that the Random Forest captures meaningful non-linear interactions between the three features—interactions that a linear model cannot represent without manual feature engineering.

Feature importance analysis reveals that mental fatigue score accounts for approximately 85% of the model’s predictive power, with resource allocation contributing around 10% and designation approximately 5%. This distribution is consistent with domain knowledge: an employee’s psychological state is the most direct indicator of burnout risk, while workload intensity and seniority play secondary roles. The strong dominance of mental fatigue score also validates the system’s architecture, which places the CBI quiz—the source of the mental fatigue score—at the centre of the assessment workflow.

The residual analysis (Figure 12) shows that prediction errors are approximately randomly distributed around zero with no systematic pattern, indicating that the model does not exhibit consistent over- or under-prediction across the burn rate range. The residual distribution is approximately normal, centred at zero, which is a desirable

property for a regression model.

Compared to public Kaggle notebooks that train on the same dataset with all available features, the system's R^2 of 0.90 using only three features falls within the reported range of 0.90–0.93 for Random Forest models. This suggests that the three selected features capture the majority of the predictive signal, and the additional columns (Gender, Company Type, WFH Setup) contribute relatively little to burn rate prediction.

5.2 Functional Testing

The system was tested end-to-end through manual testing of all user flows. Table 7 summarises the key test cases and their outcomes.

Table 7: Functional test cases and results.

Test Case	Expected Behaviour	Result
Employee registration with valid data	User created, HR profile generated, JWT returned	Pass
Registration with duplicate email	409 Conflict error returned	Pass
Login with valid credentials	JWT issued, redirect to dashboard	Pass
Login with wrong password	401 Unauthorized error	Pass
Fetch CBI quiz questions	13 questions with sections and options returned	Pass
Submit quiz with all 13 answers	Scores calculated, AI called, result saved	Pass
Submit quiz with missing answers	400 Validation error	Pass

Test Case	Expected Behaviour	Result
Reverse scoring (question 10)	$\text{adjustedValue} = 100 - \text{raw-Value}$	Pass
Mental fatigue score calculation	Correct value per Equation 4	Pass
AI prediction returns valid burn rate	Value in $[0, 1]$ range	Pass
Risk level mapping	Correct threshold applied	Pass
Alert generated on high risk	<code>high_risk</code> alert created	Pass
Alert generated on trend spike	<code>trend_spike</code> alert ($\text{delta} \geq 0.15$)	Pass
Employee dashboard data	Profile, gauge, trend, section scores	Pass
Admin dashboard aggregation	Dept. stats, risk distribution, KPIs	Pass
Department drill-down with filters	Correct filtering and sorting	Pass
Department analytics charts	All chart data populated	Pass
Employee detail (admin view)	KPI and burn rate data displayed	Pass
Mark alert as read	Alert status updated, cache invalidated	Pass
Resource CRUD (create, update, delete)	Resource persisted, table refreshed	Pass
Route guard (employee accessing admin)	Redirect to employee dashboard	Pass

Test Case	Expected Behaviour	Result
Route guard (unauthenticated access)	Redirect to login page	Pass
JWT expiry handling	401 response, redirect to login	Pass

All 23 test cases passed, confirming that the core workflows function as designed. Testing covered authentication, quiz scoring, AI integration, dashboard aggregation, alert generation, CRUD operations, and access control.

5.3 System Feature Evaluation

Table 8 evaluates each objective defined in Section 1.4 against the implemented system features.

Table 8: Evaluation of system objectives.

Obj.	Description	Status
1	Implement CBI-based assessment with mental fatigue scoring	Fully implemented
2	Train ML model for burn rate prediction	Achieved ($R^2 = 0.90$)
3	Build employee self-monitoring portal	Fully implemented (4 pages)
4	Build admin command centre with analytics	Fully implemented (6 pages)
5	Implement automated risk alerting	Fully implemented (3 alert types)

Obj.	Description	Status
6	Demonstrate microservice architecture	Fully implemented (3 services)

All six objectives have been achieved. The employee portal provides comprehensive self-monitoring with gauge visualisations, trend charts, and support resources. The admin portal offers department-level heatmaps, risk distribution analytics, KPI correlation views, and a fully functional alert inbox. The microservice architecture cleanly separates the ML concern from the web backend.

5.4 Answering the Research Questions

RQ1: How effectively can a Random Forest regression model predict employee burn rate using only three input features?

The Random Forest model achieves an R^2 of 0.90 with an MAE of 0.048, demonstrating that three features—designation, resource allocation, and mental fatigue score—are sufficient for effective burn rate prediction. The model’s performance is comparable to published results that use the full feature set, indicating that the three selected features capture the dominant signal in the data.

RQ2: Can a web-based system provide actionable burnout insights to both employees and HR administrators?

The employee portal provides personalised burn rate gauges, historical trend charts, section score breakdowns, and contextual help resources, enabling workers to monitor their own mental health proactively. The admin portal provides department heatmaps, risk distribution charts, KPI-vs-burn-rate correlations, and automated alerts,

giving HR managers actionable visibility into organisational burnout patterns. The functional testing results confirm that all data flows correctly from quiz submission through prediction to dashboard visualisation.

RQ3: What are the practical benefits and trade-offs of using a microservice architecture to separate ML inference from the application backend?

The microservice architecture provides three concrete benefits: (1) technology heterogeneity—Python’s scikit-learn ecosystem for ML and JavaScript’s Express ecosystem for web logic are each used where they excel; (2) independent development—the AI service can be retrained and redeployed without modifying or restarting the backend; (3) the mock/real switch (AI_USE MOCK) enables fully functional development without running the Python service. The primary trade-off is increased operational complexity: two servers must be running and communicating, and network failures between them require timeout handling and error propagation.

5.5 Comparison with Related Work

As summarised in Table 1, this system is the only solution among the reviewed works that combines all four elements: a trained ML model, a deployed web application, organisational-level analytics, and a validated burnout instrument (CBI).

Pradhan and Jena’s framework [9] uses machine learning for burnout classification but lacks a web deployment and does not provide organisational dashboards. Commercial platforms like Culture Amp [29] offer web-based analytics but rely on general-purpose engagement surveys without ML prediction. Kaggle community notebooks [15] demonstrate model training on the same dataset but do not integrate predictions into a usable application.

This thesis fills the integration gap by unifying psychometric assessment, machine

learning prediction, and multi-level analytics into a single platform that serves both individual workers and organisational decision makers.

CHAPTER VI

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This thesis presented the design, implementation, and evaluation of an intelligent web-based framework for the early detection of employee burnout in industrial workplaces. The system integrates a validated psychometric instrument (the Copenhagen Burnout Inventory), a machine learning prediction model (Random Forest regression), and a full-stack web application with role-based portals for both employees and administrators.

The Random Forest model, trained on approximately 22,000 records from a public dataset, achieves an R^2 of 0.90 using only three input features, demonstrating that effective burn rate prediction is possible from a minimal feature set. The employee portal empowers workers with self-monitoring tools—burn rate gauges, trend charts, and contextual support resources—while the admin portal provides organisational-level visibility through department heatmaps, risk distribution analytics, KPI correlations, and automated alerts.

The microservice architecture successfully separates the machine learning concern (Python/FastAPI) from the web application backend (Node.js/Express), enabling independent development and deployment. End-to-end functional testing confirmed that all core workflows operate correctly, from quiz submission and scoring through AI prediction to dashboard visualisation and alert generation.

6.2 Contributions

This thesis makes the following contributions:

1. **An integrated burnout detection platform** that combines psychometric assessment, machine learning prediction, and multi-level organisational analytics within a single web-based system—a combination not found in any of the reviewed related works.
2. **A practical CBI-to-ML pipeline** that transforms 13-question Copenhagen Burnout Inventory responses into a continuous burn rate prediction through a clearly documented scoring and feature engineering process, including reverse scoring, section averaging, and normalisation.
3. **A demonstrable microservice pattern** for integrating machine learning into web applications, with a clean mock/real switch mechanism that enables independent development of the ML and web components.
4. **A comprehensive admin analytics suite** including department heatmaps, risk distribution charts, KPI-vs-burn-rate correlations, and automated three-type alert generation (high risk, critical risk, and trend spike).
5. **A complete design system and component library** with 15 reusable UI primitives and 12 data visualisation components, including a custom SVG burn rate gauge.

6.3 Achievement of Objectives

All six objectives defined in Section 1.4 have been fully achieved:

1. The CBI-based assessment is implemented as a two-step wizard with proper reverse scoring and mental fatigue score derivation.

2. The Random Forest model achieves $R^2 = 0.90$, $MAE = 0.048$, and $RMSE = 0.061$, significantly outperforming the Linear Regression baseline.
3. The employee portal includes four fully functional pages: dashboard, assessment, results, and history.
4. The admin portal includes six pages: command centre, department detail (with employees and analytics tabs), company-wide employee list, individual employee detail, alerts inbox, and resource management.
5. Three alert types (high risk, critical risk, and trend spike) are automatically generated and displayed in a filterable inbox with mark-read functionality.
6. The three-service architecture (React/Next.js, Node.js/Express, Python/FastAPI) is fully operational with mock/real mode switching.

6.4 Limitations

Several limitations should be acknowledged:

- **Simulated HR data.** Designation and resource allocation values are randomly generated at registration rather than sourced from a real factory HR system. While values are drawn from the training dataset's ranges, they do not reflect actual workplace conditions.
- **Limited feature set.** The model uses only three features. Including additional signals such as actual shift patterns, overtime records, or longitudinal CBI trends could improve predictive accuracy.
- **Single dataset.** Model training and evaluation rely on a single public Kaggle dataset. The model's performance may not generalise across different industries, cultures, or organisational contexts.

- **Simulated KPI data.** The employee KPI records (attendance, productivity, quality) are seeded rather than collected from real workplace systems, limiting the validity of the KPI-vs-burn-rate correlation analysis.
- **Desktop-only interface.** The application is designed for minimum 992 px screen widths. Factory workers using personal mobile devices cannot access the system.
- **No formal usability study.** The system was tested functionally but not evaluated through a structured usability study with real end users.
- **No real-time updates.** The admin dashboard uses polling-based data fetching via TanStack Query rather than real-time push notifications via WebSocket or Server-Sent Events.

6.5 Future Work

The following directions are proposed for future development:

- **Mobile responsive design.** Extend the frontend with responsive breakpoints to support mobile and tablet access, enabling factory workers to complete assessments on personal devices.
- **Real-time alerts.** Implement WebSocket or Server-Sent Events to push new alerts to the admin dashboard instantly, without requiring page refresh or polling intervals.
- **Integration with real HR systems.** Replace the simulated HR profile generator with connectors to actual HR information systems (e.g., SAP SuccessFactors, BambooHR) to provide authentic designation and workload data.
- **Advanced ML models.** Explore Gradient Boosting (XGBoost, LightGBM)

and time-series models that incorporate longitudinal CBI trends, enabling the system to predict burn rate trajectories rather than point estimates.

- **Expanded feature engineering.** Incorporate additional features such as actual overtime hours, shift rotation frequency, and seasonal patterns to improve model accuracy.
- **Export functionality.** Add CSV and PDF export for department analytics reports and assessment history, enabling offline review and reporting.
- **Accessibility improvements.** Add ARIA labels to custom components (gauge, heatmap) and conduct automated accessibility testing to ensure compliance with WCAG guidelines.
- **Multi-language support.** Internationalise the user interface to support Vietnamese and other languages relevant to the target industrial workforce.
- **Formal usability evaluation.** Conduct a structured usability study with factory workers and HR personnel to validate the system's effectiveness and identify user experience improvements.

LIST OF REFERENCES

- [1] World Health Organization, “Burn-out an ‘occupational phenomenon’: International Classification of Diseases,” WHO, May 2019. [Online]. Available: <https://www.who.int/news/item/28-05-2019-burn-out-an-occupational-phenomenon-international-classification>
- [2] C. Maslach, W. B. Schaufeli, and M. P. Leiter, “Job burnout,” *Annual Review of Psychology*, vol. 52, no. 1, pp. 397–422, 2001.
- [3] C. Maslach and S. E. Jackson, “The measurement of experienced burnout,” *Journal of Organizational Behavior*, vol. 2, no. 2, pp. 99–113, 1981.
- [4] H. J. Freudenberger, “Staff burn-out,” *Journal of Social Issues*, vol. 30, no. 1, pp. 159–165, 1974.
- [5] T. S. Kristensen, M. Borritz, E. Villadsen, and K. B. Christensen, “The Copenhagen Burnout Inventory: A new tool for the assessment of burnout,” *Work & Stress*, vol. 19, no. 3, pp. 192–207, 2005.
- [6] B. Wigert and S. Agrawal, “Employee burnout, Part 1: The 5 main causes,” Gallup Workplace, 2020. [Online]. Available: <https://www.gallup.com/workplace/237059/employee-burnout-part-main-causes.aspx>
- [7] K. Ahola, A. Toppinen-Tanner, and J. Seppänen, “Interventions to alleviate burnout symptoms and to support return to work among employees with burnout: System-

- atic review and meta-analysis,” *Burnout Research*, vol. 4, pp. 1–11, 2017.
- [8] R. Bianchi, I. S. Schonfeld, and E. Laurent, “Burnout: Moving beyond the status quo,” *International Journal of Stress Management*, vol. 26, no. 1, pp. 36–45, 2019.
- [9] R. K. Pradhan and L. K. Jena, “Employee performance at workplace: Conceptual model and empirical validation,” *Business Perspectives and Research*, vol. 5, no. 1, pp. 69–85, 2017.
- [10] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009.
- [12] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [13] A. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim, “Do we need hundreds of classifiers to solve real world classification problems?” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 3133–3181, 2014.
- [14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [15] V. Subhash, “Employee Burnout Prediction,” Kaggle, 2021. [Online]. Available: <https://www.kaggle.com/datasets/vijaysubhashp/employee-burnout-prediction>

- [16] Meta Platforms, Inc., “React — A JavaScript library for building user interfaces,” 2024. [Online]. Available: <https://react.dev/>
- [17] Vercel, Inc., “Next.js — The React Framework,” 2024. [Online]. Available: <https://nextjs.org/>
- [18] OpenJS Foundation, “Node.js,” 2024. [Online]. Available: <https://nodejs.org/>
- [19] OpenJS Foundation, “Express — Fast, unopinionated, minimalist web framework for Node.js,” 2024. [Online]. Available: <https://expressjs.com/>
- [20] S. Ramírez, “FastAPI,” 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [21] T. Linsley, “TanStack Query (React Query),” 2024. [Online]. Available: <https://tanstack.com/query/latest>
- [22] Tailwind Labs, “Tailwind CSS,” 2024. [Online]. Available: <https://tailwindcss.com/>
- [23] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2015.
- [24] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, 2000.

- [25] Oracle Corporation, “MySQL 8.0 Reference Manual,” 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [26] MongoDB, Inc., “MongoDB Documentation,” 2024. [Online]. Available: <https://www.mongodb.com/docs/>
- [27] C. Burdo, “Zod — TypeScript-first schema validation,” 2024. [Online]. Available: <https://zod.dev/>
- [28] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
- [29] Culture Amp Pty Ltd, “Culture Amp — Employee Experience Platform,” 2024. [Online]. Available: <https://www.cultureamp.com/>
- [30] Recharts Group, “Recharts — A composable charting library built on React components,” 2024. [Online]. Available: <https://recharts.org/>

APPENDICES

APPENDIX A: COMPLETE API REFERENCE

Table 9 provides the complete list of RESTful endpoints exposed by the Express backend and the Python AI microservice.

Table 9: Complete API endpoint reference.

Method	Endpoint	Access	Purpose
<i>Authentication</i>			
POST	/api/auth/register	Public	Create account + HR profile
POST	/api/auth/login	Public	Authenticate, get JWT
GET	/api/auth/me	Any	Get current user profile
<i>Quiz</i>			
GET	/api/quiz	Employee	Fetch CBI quiz questions
POST	/api/quiz/submit	Employee	Submit answers, get prediction
GET	/api/quiz/history	Employee	Past submissions (paginated)
<i>Employee Dashboard</i>			
GET	/api/dashboard/employee	Employee	Full dashboard data
GET	/api/resources	Employee	Help resources for risk level
<i>Admin Dashboard</i>			
GET	/api/dashboard/admin	Admin	Command centre overview

Method	Endpoint	Access	Purpose
GET	/api/dashboard/admin/ department/:id	Admin	Department drill-down (sort/filter/paginate)
GET	/api/dashboard/admin/ department/:id/analytics	Admin	Department analytics
GET	/api/dashboard/admin/ employees	Admin	Company-wide employee list
GET	/api/dashboard/admin/ employees/:userId	Admin	Single employee detail
<i>Alerts</i>			
GET	/api/alerts	Admin	List alerts (filterable)
PATCH	/api/alerts/:id/read	Admin	Mark single alert as read
PATCH	/api/alerts/read-all	Admin	Mark all alerts as read
<i>Admin Resource Management</i>			
GET	/api/admin/resources	Admin	List all help resources
POST	/api/admin/resources	Admin	Create a help resource
PUT	/api/admin/resources/:id	Admin	Update a help resource
DELETE	/api/admin/resources/:id	Admin	Delete a help resource
<i>AI Microservice (internal)</i>			
POST	/api/predict	Internal	Predict burn rate
GET	/api/health	Internal	AI service health check

All responses use the JSON envelope format:

Success: { "success": true, "data": { ... } }

Error: { "success": false, "error": { "code": "...", "message": "..." } }

APPENDIX B: DATABASE SCHEMA

B.1 MySQL Tables

```
CREATE TABLE users (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    email         VARCHAR(255) UNIQUE NOT NULL,  
    password_hash VARCHAR(255) NOT NULL,  
    first_name    VARCHAR(100) NOT NULL,  
    last_name     VARCHAR(100) NOT NULL,  
    gender        ENUM('Male','Female','Other') NOT NULL,  
    date_of_birth DATE,  
    role          ENUM('employee','admin') NOT NULL DEFAULT 'employee',  
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
                ON UPDATE CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE departments (  
    id            INT AUTO_INCREMENT PRIMARY KEY,  
    name          VARCHAR(100) UNIQUE NOT NULL,  
    location      VARCHAR(100),  
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE hr_profiles (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT UNIQUE NOT NULL,
    department_id     INT NOT NULL,
    date_of_joining   DATE NOT NULL,
    company_type      ENUM('Service','Product') NOT NULL,
    wfh_available     BOOLEAN NOT NULL DEFAULT FALSE,
    designation       INT NOT NULL,
    resource_allocation INT,
    shift_type        ENUM('Day','Night','Rotating')
                    NOT NULL DEFAULT 'Day',
    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                    ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (department_id)
        REFERENCES departments(id) ON DELETE CASCADE
);

```

```

CREATE TABLE assessment_results (
    id                INT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT NOT NULL,
    personal_burnout_score FLOAT,
    work_burnout_score    FLOAT,

```

```

    mental_fatigue_score    FLOAT NOT NULL,
    predicted_burn_rate     FLOAT NOT NULL,
    risk_level              ENUM('low','moderate','high','critical')
                           NOT NULL,
    taken_at                TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_history (user_id, taken_at)
);

```

```

CREATE TABLE alerts (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    assessment_result_id   INT NOT NULL,
    user_id                INT NOT NULL,
    alert_type             ENUM('high_risk','critical_risk',
                              'trend_spike') NOT NULL,
    message                TEXT NOT NULL,
    is_read                BOOLEAN NOT NULL DEFAULT FALSE,
    created_at             TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (assessment_result_id)
        REFERENCES assessment_results(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_unread_alerts (is_read, created_at)
);

```

```

CREATE TABLE employee_kpis (

```



```

id            INT AUTO_INCREMENT PRIMARY KEY,
user_id       INT NOT NULL,
period_date   DATE NOT NULL,
attendance_rate  FLOAT NOT NULL,
productivity_score FLOAT NOT NULL,
overtime_hours  FLOAT NOT NULL,
tasks_completed INT NOT NULL,
quality_score  FLOAT NOT NULL,
days_absent   INT NOT NULL,
created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
UNIQUE INDEX uq_kpi_user_period (user_id, period_date),
INDEX idx_kpi_period (period_date)
);

```

```

CREATE TABLE help_resources (
id            INT AUTO_INCREMENT PRIMARY KEY,
title        VARCHAR(200) NOT NULL,
description   TEXT,
url          VARCHAR(500),
min_risk_level ENUM('low','moderate','high','critical') NOT NULL,
is_active    BOOLEAN NOT NULL DEFAULT TRUE,
created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
INDEX idx_active_risk (is_active, min_risk_level)
);

```

B.2 MongoDB Collections

Collection: `cbi_questions` — Singleton document containing the complete CBI quiz definition.

```
{
  "_id": ObjectId,
  "name": "Copenhagen Burnout Inventory (excerpt)",
  "version": 1,
  "isActive": true,
  "responseOptions": [
    {"label": "Always", "value": 100},
    {"label": "Often", "value": 75},
    {"label": "Sometimes", "value": 50},
    {"label": "Seldom", "value": 25},
    {"label": "Never/almost never", "value": 0}
  ],
  "sections": [
    {"id": "personal_burnout",
     "title": "Personal burnout",
     "questionIds": [1,2,3,4,5,6]},
    {"id": "work_related_burnout",
     "title": "Work-related burnout",
     "questionIds": [7,8,9,10,11,12,13]}
  ],
  "questions": [ /* 13 question objects */ ]
}
```

```
}
```

Collection: quiz_submissions — One document per assessment attempt.

```
{
  "_id": ObjectId,
  "mysqlAssessmentId": 42,
  "mysqlUserId": 1,
  "quizVersion": 1,
  "responses": [
    {"questionId": 1, "answerLabel": "Often",
     "answerValue": 75},
    {"questionId": 10, "answerLabel": "Seldom",
     "rawValue": 25, "adjustedValue": 75,
     "reverseScored": true},
    ...
  ],
  "personalBurnoutAvg": 58.33,
  "workBurnoutAvg": 64.29,
  "submittedAt": ISODate("2026-05-03T10:30:00Z")
}
```

APPENDIX C: COPENHAGEN BURNOUT INVENTORY QUESTIONNAIRE

The system uses 13 questions from the Copenhagen Burnout Inventory, covering two sections. Response options for all questions are: Always (100), Often (75), Sometimes (50), Seldom (25), Never/almost never (0).

C.1 Personal Burnout (Questions 1–6)

1. How often do you feel tired?
2. How often are you physically exhausted?
3. How often are you emotionally exhausted?
4. How often do you think: “I can’t take it anymore”?
5. How often do you feel worn out?
6. How often do you feel weak and susceptible to illness?

C.2 Work-Related Burnout (Questions 7–13)

7. Do you feel worn out at the end of the working day?
8. Are you exhausted in the morning at the thought of another day at work?
9. Do you feel that every working hour is tiring for you?
10. Do you have enough energy for family and friends during leisure time?
(Reverse-scored: $adjustedValue = 100 - rawValue$)
11. Is your work emotionally exhausting?
12. Does your work frustrate you?
13. Do you feel burnt out because of your work?

C.3 Scoring

1. Calculate the personal burnout average: mean of questions 1–6 (range 0–100).
2. Calculate the work-related burnout average: mean of questions 7–13 (range 0–100), with question 10 reverse-scored.
3. Calculate the mental fatigue score: $\text{mentalFatigueScore} = \frac{\text{personalAvg} + \text{workAvg}}{20}$ (range 0.0–10.0).